École Polytechnique Fédérale de Lausanne
University of California Berkeley

Mechanical Engineering

Laboratoire d'Automatique
Model Predictive Control Laboratory

Master Project

On the Implementation of an Active Learning Model Predictive Controller: An Autonomous Race Car Example

Author:
Martin d'Hoffschmidt

Supervision:
Ugo Rosolia
Prof. Francesco Borrelli
Prof. Colin Neil Jones

Handed in to Prof. Colin Neil Jones on the 6th of November 2016.

Academic year 2016-2017

Martin

d. Hoffschmidt

Contents

1 Introduction **1**
1.1 Structure of the work . . . 2
1.2 Other works . . . 3

2 The race car system **4**
2.1 Problem statement . . . 4
2.1.1 Modelling assumptions . . . 5
2.1.2 Challenges . . . 5
2.2 Race car dynamics . . . 7
2.2.1 Definitions . . . 7
2.2.2 The lateral forces . . . 8
2.2.3 The longitudinal forces . . . 9
2.2.4 The "g-g" diagram . . . 10
2.3 System identification . . . 11
2.3.1 Generalized car dynamics . . . 11
2.3.2 Modelling limitations . . . 13
2.3.3 Learning model . . . 14

3 System identification **15**
3.1 Definitions . . . 15
3.1.1 Features and targets . . . 15
3.1.2 Data preprocessing . . . 16
3.1.3 Supervised learning problem . . . 16
3.1.4 Regularization . . . 17
3.1.5 Performance measure . . . 17
3.2 Linear regression . . . 18
3.2.1 Choosing the features . . . 18
3.2.2 Training . . . 19
3.3 Neural Networks . . . 20
3.3.1 Definitions . . . 20
3.3.2 Reconstruction layer . . . 21
3.3.3 Squashing function . . . 22
3.3.4 Gradient descent optimization methods . . . 23
3.3.5 Parameter initialization . . . 25
3.4 Learning strategy . . . 26
3.4.1 Semi-incremental learning . . . 26
3.4.2 Incremental learning . . . 27

4 Curiosity **28**
4.1 Theory of curiosity . . . 28
4.1.1 Intrinsic motivation and external goal . . . 28

4.1.2 General the intrinsic cost . . . 29
4.2 Performance of the adaptive model . . . 30
4.2.1 Performance evaluator . . . 30
4.2.2 Performance of the model . . . 31
4.3 Curiosity cost . . . 32
4.3.1 Curiosity as the model error . . . 32
4.3.2 Curiosity as improvement of the model . . . 33
4.3.3 Curiosity as changes in model predictions . . . 33
4.4 Predicting the curiosity . . . 33

5 The controller **35**
5.1 Problem formulation . . . 35
5.2 Iterative learning control . . . 36
5.2.1 Definitions and assumptions . . . 36
5.2.2 Final safe set . . . 37
5.2.3 Final cost . . . 38
5.2.4 Learning MPC problem . . . 38
5.3 Incremental curiosity-driven learning model predictive control . . . 39

6 Open-loop simulations **41**
6.1 Cross-validation . . . 41
6.1.1 Cross-validation of the model . . . 42
6.1.2 Cross-validation of the learning strategy . . . 42
6.2 Linear regression . . . 42
6.2.1 Learning curves . . . 43
6.2.2 Validation curve . . . 43
6.2.3 Simulation results . . . 44
6.3 Neural network . . . 44
6.3.1 Choice of the sigmoid . . . 45
6.3.2 Choice of the solver . . . 45
6.3.3 Learning curves . . . 46
6.3.4 Validation Curve . . . 47
6.3.5 Simulation and comparison . . . 47
6.4 Learning strategy . . . 48

7 Closed-loop simulations **51**
7.1 Simulation set-up . . . 51
7.1.1 Description of the track . . . 51
7.1.2 Details for the controller . . . 52
7.1.3 Details for the predictors . . . 52
7.2 Simulation for the learning strategy . . . 52
7.2.1 Performance of the controller . . . 53
7.2.2 Evolution of the velocity profile . . . 53
7.3 Simulation for the curiosity cost . . . 54
7.3.1 Influence of active learning on the performance of the controller . . . 54
7.3.2 Influence of active learning on the race-car behaviour . . . 55

8 Conclusion **57**

List of Figures

2.1 Illustration of the states z for the race car 5
2.2 Illustration of the speed profile of a race car on a circuit [21] 6
2.3 Example of "g-g" measurement diagram during a Grand Prix [21] 6
2.4 Illustration of the simplified tire model 7
2.5 Lateral tire force as a function of the slip ratio for various slip angles [30] . . 9
2.6 Lateral tire force F_y versus slip angle α_s for several tractive slip ratios S_t [30] 9
2.7 Lateral tire force F_y versus slip angle α_s for several braking slip ratios S_b [30] 9
2.8 Longitudinal tire force F_x as a function of the slip ratio αs for various slip angles S_t and S_b [30] 10
2.9 Illustration of the generalized car model in the horizontal plane 11

3.1 Illustration of a fully connected neural network 21
3.2 Illustration of the bias unit in neural networks 21
3.3 Illustration of the hidden layers and reconstruction layer 22
3.4 Illustration of the squashing functions 22

5.1 Illustration of the MPC controller 40

6.1 Illustration of the forward velocity v_x for several laps 41
6.2 Learning curves for the linear regression model 43
6.3 Validation curve for the linear regression model 44
6.7 Learning curves for the neural network model 46
6.8 Validation curve for the neural network model 47
6.9 Training curve for the neural network models, open-loop simulation 47
6.11 Learning strategy cross-validation for the longitudinal velocity model 49

7.1 Illustration of the track used for the simulations 51
7.2 Performance of the controller for two learning strategies 53
7.4 Comparison of the curiosity methods and learning methods, closed-loop simulation 54
7.5 Longitudinal speed profile of the race car for the controller with and without active learning 55
7.6 Illustration of the "g-g" diagram for the simulation with and without active learning 55
7.7 Lateral position of the race car on the track for the controller with and without active learning 56

Abstract

In this Master Thesis, a novel control framework is proposed to push the performance of an autonomous racing car driving on a circuit. In order to achieve this goal, the possibility of combining model based control strategies with machine learning techniques is explored. The control problem is reformulated as an iterative task, where the vehicle performs several laps on the track and the data from the previous laps are used to design a controller able to minimize the vehicle's lap time.

The above control problem is challenging as *i)*: to compute the optimal trajectory one would need to solve an optimal control problem over the track length, which is computationally challenging, especially in real-time, *ii)*: a highly non-linear vehicle is needed to push the performance, as the controller should operate the vehicle at the limit of its handling capability.

To tackle these problems, one uses the Learning Model Predictive Control (LMPC) framework based on a model learnt from data. The LMPC used the data from the previous laps to compute a invariant control set and an approximation of the cost-to-go that allow to overcome the problem related with the short horizon of the receding control strategy. However, the LMPC relies on a vehicle model that is generally hard to identify. In this work, two different system identification strategies are proposed to learn the plant model. First, a linear regressor is implemented to identify the lateral and longitudinal vehicle dynamics. Afterwards, a fully connected neural network is used as a plant model. It is shown that for both approaches the LMPC combined with the system identification techniques is able to improve the closed loop performance until convergence is achieved.

Finally, one proposes a "curiosity" or "exploration" strategy. The LMPC uses the data from previous iterations to improve the performance and it converges to a local minima of the optimal control problem. A curiosity cost is therefore introduced such as to force the controller to explore actively different region of the states space even thought this leads to a worsening of the overall system performance. One shows that the LMPC augmented with the curiosity cost outperforms the standard LMPC design and it is able to find a better performing trajectory for the examined control problem. The above described curiosity or exploration strategy is the core contribution of this work.

Acknowledgement

I would like to thank warmly Ugo Rosolia for the quality of his supervision throughout this project, for the very useful and relevant comments and suggestions to improve the present work and for his significant help and contribution in the implementation of the simulations.

Chapter 1

Introduction

The race car drivers want to be the first to cross the finish line, and their ability to achieve this goal is determined by their driving skills based on experience. The performance of the driver on the track depends on its ability to push the vehicle towards its limits without losing the control. This knowledge about the vehicle is acquired through an intense training. While the final goal is clear, the strategy that each driver uses to achieve it is more subjective. It can be considered as a combination of technical knowledge, intuition and experience. For instance, a skilled driver is able to feel the behaviour of the car in a corner perceiving the limits of what the vehicle can achieve. The focus of the present work consist in exploring how it is possible for an autonomous race car to improve its skills on the track, similarly to a human race driver.

Humans can in practice solve, easily and intuitively, a very large range of complex problems. From standing up and walking, swimming, cooking or singing, acquiring new skills is made possible through a complex learning process. An individual learns a specific task through repetitive trials. Which leads him ultimately to improve its skills at performing this task. The learning process involved in improving a specific skill can be characterized by two main aspects. First, the learning process is carried out in a continuous fashion. Indeed, the individual never stops learning. The ability to learn may eventually decay over time, which leads to the second aspect. An individual that is motivated or that is having fun at performing a given task is likely to learn faster. Whereas, he may turn away from the task when she appears to be boring or too complex.

In the present work, the autonomous race car is controlled with a Learning Model Predictive Controller (LMPC). In this way, the racing objective is clearly defined and the constraints of the race car can explicitly be handled. Improving the performance of the controller can be achieved in two different ways. First, the repetitive aspect of racing on a circuit makes it possible to integrate additional knowledge on the task. By analogy, the more laps the race driver completes the better he knows the tricks to improve its lap time. Second, it is possible to identify a model of the car, while driving, to always improve the model that is provided to the controller. Which can be compared to the ability of the driver to improve the knowledge he has on the vehicle to acquire a better intuition on its dynamics. Finally, a curiosity cost is added to the objective function of the optimal controller to motivate the controller at improving its model.

1.1 Structure of the work

The first chapter states the problem and the challenges of autonomous racing. Given the assumptions done on the states and the inputs of the model, it possible to understand better which dynamics can be captured and which cannot. The accuracy or reliability of the model impacts the performance of the controller. This is illustrated through a specific analysis tool referred to as the lateral and longitudinal acceleration diagram. Which is used in practice to evaluate the performances of a driver on a circuit. It can be shown that there exists a close relationship between this diagram and the forces acting on the tires. The study of their behaviour is particularly interesting as they provide the main source of control and stability to the vehicle. From there, the limitations of a commonly used handwritten model can be identified. Finally, one presents how a better model can be learnt from collected data samples.

The second chapters presents the two different approaches to identify the dynamics of the race car. First, linear regression can be very powerful to learn a model. Indeed, the system identification problem can then be posed as a convex optimization problem. Very efficient methods are available to solve such problems. Second, the model can be represented by a neural network. While neural networks are recognized to be good general non-linear learners, the resulting system identification problem yields a non-convex optimization problem to solve, which is subject to local minima. To tackle this problem, some heuristics and specific gradient-based methods can be used and are described in this work. Finally, one presents how it is possible to train a model in an incremental and semi-incremental way. The two learning strategies solve the problem of a once-and-for-all learned model. Indeed, as new laps are completed, new data become available to further improve the model.

The system identification process is usually carried out in a passive way. However, when the system identification is carried out incrementally, or in closed-loop, it is possible to design a controller that actively excite the unmodelled system dynamics. In this way, significant data are collected for the system identification process. This is referred to as active learning [37]. The approach followed in the present work builds on the one proposed by Schmidhuber in [34] and [35]. The idea is that the controller should steer the race car towards areas of the environment which lead to improvements of the model. For this purpose, a curiosity cost is added to the controller, which determines the best trade-off between using the model to achieve his goal or train the model to reach, ultimately, better performances. Three ways to compute the curiosity cost are presented and discussed. Finally, it shown how another model, predicting future improvement of the vehicle model, can be trained and provided to the Model Predictive Controller.

The fourth chapter presents the Learning Model Predictive Controller (LMPC). First, the choice for the cost function is motivated, the constraints on the vehicle are discussed and the formulation of the problem as an infinite horizon problem is presented. Second, from the work of Rosolia [28], it is indicated how it is possible at the same time to solve the infinite horizon problem for a non-linear application and guarantee recursive feasibility of the problem as well as non-increasing cost performance. Third, more details are given on the implementation of a learning model predictive controller driven by curiosity.

The fifth chapter presents open-loop simulations results to evaluate the performance of the model at predicting future states of the race car. First, the linear regression model is evaluated through the analysis of the learning and validation curves and simulation results are shown. Second, the neural network model is evaluated in exactly the same way. In addition, one investigates how the choice for the sigmoid functions and the solver can influence the performance of the model. Finally, one presents the open-loop results obtained for both the

incremental and semi incremental learning strategies.

The sixth and last chapter presents the close-loop simulation results. The set-up and implements details of the simulation are first given and discussed. Second, two distinct simulations were carried with the NLPMC without active learning. The purpose of those is to determine how the two different learning strategies impact the performance of the controller. Third, three other simulations are performed to investigate on the influence of the three proposed methods to generate the curiosity cost. These results are then compared to the results obtained without active learning.

1.2 Other works

Linear regression has been extensively and successfully used for many system identification applications. For instance, Abbeel et al. in [1] used a least square method to model the dynamics of an helicopter. Based on these results, Punjani et al. in [25] has trained a deep neural network to improve the accuracy the helicopter model. His approach consists in learning the continuous state model. Predicting discrete states of the system has been successfully investigated by Chen et al. in [4].

Neural networks model have been used for model predictive control application as early as in the 90's. In his work, Draeger et al. [6] did successfully use a fully-connected neural network to identify the model of a neutralization reactor to control the pH. More recently, Lenz et al. in [17] used deep learning combined with model predictive control for the task of cutting vegetables. A neural network model is interesting for this application as it is difficult to hand-design a general model of the vegetable's dynamic properties. Other works such as [38] and [43] go further and use Model Predictive Control to generate data to train a control policy that maps directly that states of the system to the optimal control actions.

The theory of creativity, fun and intrinsic motivation is presented by Schmidhuber in [34] and [35]. The core idea of the theory states that a curios agent is able to promote interesting sequences of actions to systematically improve its model, which leads him to perform other tasks with higher performances. He provides also a general synchronous framework to generates curiosity rewards for reinforcement learning applications. However, this framework requires some modifications to be combined with a model predictive controller. In earlier work, [32] and [31], Schmidhuber suggested the interesting idea of training an additional model at forecasting future expected rewards of improving the model. This approach is followed in the present work because it can be very easily implemented in the objective function of the model predictive controller. Other works have developed very similar ideas on intelligent curios agents that maximize the expected learning progress of the model (e.g. [24], [18], [2]).

Finally, the present work has two main contributions. First, one proposes two simple incremental learning strategies for the race car application. For each learning strategy, the use of a linear regression and a neural network model is investigated. Improving the model continuously is likely to enhance the success of the controller as the model mismatch is reduced. Second, one proposes to train a neural network at predicting future improvements of the race car dynamic model. Coupling the theory of curiosity with model predictive control is two-fold. One the one hand, the optimal controller promotes actively the system identification process. On the other-hand, the exploration due to the curiosity cost is likely to yield sequences of trajectories that will positively impact the learning of the final cost and safe set of the controller. To our knowledge, it is the first time that such a model predictive controller framework is proposed.

Chapter 2

The race car system

In this chapter, the challenges related to the race car modelling are presented. First, the problem is stated and the assumptions on the state and input space are discussed. The need to provide a reliable model to the controller is motivated. For this purpose, one introduces the lateral-longitudinal acceleration diagram. Which is a common tool used in the racing area to evaluate the performance of a driver, or a vehicle, on a given track. The same tool can therefore be used to evaluate the performance of the controller. A commonly used model in the field of vehicle control using MPC is the bicycle model. One investigates on the limitations of the model to capture important dynamic features of the race car. It is argued that these limitations make it difficult for the controller to exploit the maximum manoeuvring capabilities of the system. Finally, one presents how this problem can be addressed using machine learning to identify a better model based on recorded data samples of vehicle.

2.1 Problem statement

The objective of the present work is to steer a race car on a circuit in an autonomous way. For this purpose, a non-linear Model Predictive Controller is used. The optimal controller should ideally manoeuvre the vehicle at the limits of its handling capabilities to improve its lap time. For this purpose, a model of the system is required. On the one hand, modelling the dynamics of the car is a multi-physic problem that involves aerodynamics, kinematics, material resistance, tire behaviour, etc. On the other hand, the ability of the model at predicting accurately future states of the system is essential to improve its manoeuvring capabilities. Due to time constraints, the right a trade-off between modelling accuracy and computational complexity must be established.

Consider a dynamical system with state vector $z \in \mathbb{R}^{n_z}$ and control input vector $u \in \mathbb{R}^{n_u}$. The number of states and inputs is respectively given by n_z and n_u. The function $f : \mathbb{R}^{n_z+n_u} \to \mathbb{R}^{n_z}$ represents the dynamics of the vehicle given by the time derivatives on the state vector $\dot{z}$ such that,

$$\dot{z} = f(z, u)\cdot \tag{2.1}$$

In other words, the main contribution of the this chapter is to investigate on the choice of the right function f to model the dynamics of the race car.

2.1.1 Modelling assumptions

It is assumed in the present work that the states z of the vehicle are given by its position, orientation, velocities and angular velocities. It is further considered that the roll and the pitch of the car can be neglected. Assuming a flat track, they should indeed always remain equal or close to zero within normal operating conditions. Under the same assumption, the altitude of the vehicle is neglected as well.

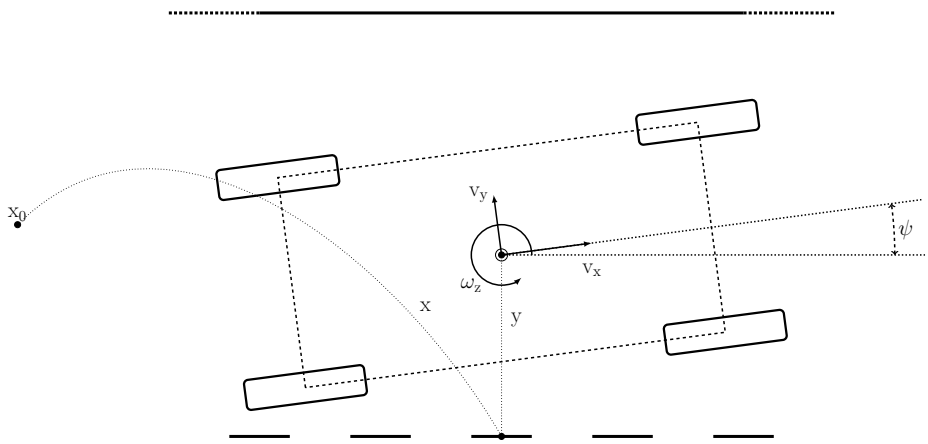


Figure 2.1: Illustration of the states z for the race car

For the race car application, it is realistic to assume that the track is known in advance. Therefore, the absolute position of the vehicle on the track can be expressed according to x and y. The position x of the vehicle on the track is expressed as the distance between the position of the car with respect to the center-line of the road and a specific reference point on the track, referred to as x_0. The position y simply denotes the distance of the car from the center line. The longitudinal and lateral velocities of the vehicle it its reference frame are respectively denoted by v_x and v_y. The yaw ψ of the car is measured with respect to the curvature of the road. Finally, the yaw rate is referred to as ω_z. The figure 2.1 gives an illustration of the vehicle's states. The state vector z is be expressed as,

$$z = \Big[x,\ y,\ \psi,\ v_x,\ v_y,\ \omega_z,\Big]^\top. \tag{2.2}$$

The three actions that can be carried out by the driver, or the controller, on the vehicle are accelerating, braking and steering. In the present work, the autonomous race car is equipped with a low-level controller that translates negative or positive input acceleration a_x into a braking or tractive action. This low level controller can be, for instance, a PID or PI. Therefore, only two quantities are necessary to control the car, namely the acceleration input a_x and the steering input δ. The input vector u can be expressed as,

$$u = [a_x, \delta]^\top. \tag{2.3}$$

2.1.2 Challenges

The purpose of racing consists in travelling from one point to another one in the minimum amount of time. For this reason, the speed profile of a race car is always changing. Whereas a usual car used for commuting exhibits rather a flat speed profile. As illustrated on the figure 2.2, the speed changes continually over the entire track as the driver enters a corner,

accelerates, brakes, etc. In order to decrease the lap time, the speed has to increase or decrease by its maximum rate of change.

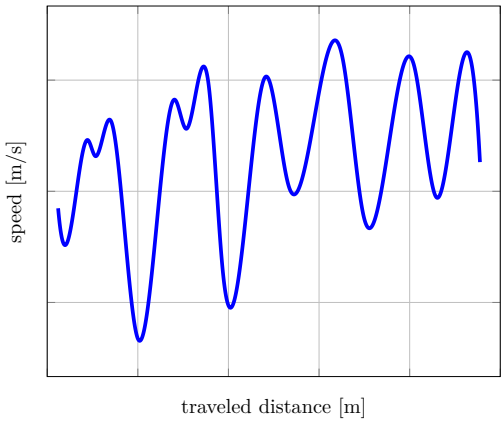


Figure 2.2: Illustration of the speed profile of a race car on a circuit [21]

A skilled race driver tries to maximize the resultant of the car's acceleration at any-time [21]. Therefore, the lateral-longitudinal acceleration diagram, or the "g-g" diagram, is a helpful analysis tool to evaluate the performance of a driver on the track. It consists in measuring and plotting the lateral and longitudinal accelerations data points on respectively the abscissa and ordinate of the same graph. For instance, figure 2.3 is the result of some measurements taken on the Paul Ricard circuit during the 1977 Grand Prix [21]. These measurements report the performances of the driver on the track. But, it is also indicative of the vehicle's performances. Figure 2.3 shows namely that the driver tries to manoeuvre the car away from the origin of the diagram.

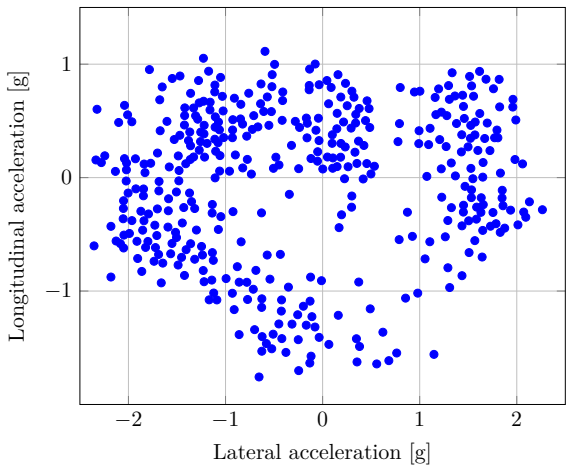


Figure 2.3: Example of "g-g" measurement diagram during a Grand Prix [21]

The goal of the race car manufacturers consist in increasing as much as possible the total available area on the "g-g" diagram [21]. The forward acceleration depends on the engine characteristics of the vehicle, aerodynamic shape, gearing etc. The braking and lateral acceleration performances depend mostly on the tires characteristics, the tire-road friction coefficients, the load balance, the suspensions etc. Because the potential boundaries of the "g-g" diagram are very difficult to determine in practice for a specific car configuration and circuit, it is almost impossible to know exactly where the true boundaries are located[21].

The goal of the race-driver is to steer the vehicle towards the boundaries of the "g-g" diagram. Doing so, the skilled driver improves its performance on the track and, consequently, decreases its lap time. However, he needs to be aware of the limits such that he do not lose the controllability of the vehicle. The goals of autonomous racing are exactly the same. The controller must also be able to manoeuvre the vehicle close to the boundaries of the "g-g" diagram. It is therefore essential to provide the controller with a reliable model of the race car such that it can, similarly to a skilled driver, feel the limits of the system.

2.2 Race car dynamics

The goal of the system identification process it to provide a reliable knowledge of the race car dynamics to the controller in such a way that it is able to manoeuvre the vehicle near the boundaries of the "g-g" diagram. One focusses here in understanding how the driver and the controller can act on the system to achieve this challenge.

The tires and the aerodynamic effects contribute mostly to the performances of the race car. The tires are the source of forces and torques for accelerating, braking and steering the car [21]. They provide the car control and stability. The aerodynamic effects are responsible for drag and aero down-force which improve braking and adherence of the vehicle on the track. The fundamental difference between the tires and the aerodynamic effect is that, in general, the first one can actively be controlled by the driver while the second one cannot. Indeed, the driver can actively modify the states of the car by accelerating, braking and/or steering, which affects the forces acting on the tires. Whereas the aerodynamic effects can in principle not be modified during a race. Which is at least the case in the present work. For this reason, it is reasonable to focus essentially on the behaviour of the tires.

2.2.1 Definitions

In a very simplified way, it is possible to take into consideration only the forces acting on the tire in the horizontal plane of contact at the tire-road interface. The longitudinal and lateral forces are respectively denoted by F_x and F_y. The longitudinal force is responsible for braking and accelerating the tire. The lateral force is responsible for turning the tire. Figure 2.4 gives a simple illustration of the tire model. The attention is drawn to the two main aspects that affect the tire at producing a longitudinal and/or a lateral force, namely the slip angle and the slip ratio.

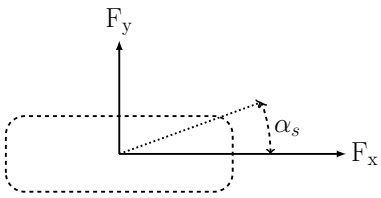


Figure 2.4: Illustration of the simplified tire model

The slip angle is defined as the angle between the heading of the tire and the direction it is actually going to. Referring to figure 2.4, the slip angle α_s can be defined as,

$$\alpha_s = \arctan\left(\frac{V_y}{|V_x|}\right), \tag{2.4}$$

where V_x and V_y are respectively the longitudinal and lateral velocity of the tire in the horizontal plane.

The accelerating or braking tires of a car don't have the same velocity has a free rolling wheel would have. A free rolling wheel corresponds to a wheel rolling on a surface without the presence of slipping effects. In fact, the motion of the tire on the road is a combination of rolling, sliding and slipping. The difference between the angular velocity of the free rolling wheel and the true angular velocity of the wheel defines the slip ratio [21],

$$S_r = \frac{\Omega - \Omega_0}{\Omega_0}, \tag{2.5}$$

where $\Omega = V/R_e$ is defined as the angular speed of the free rolling wheel. The effective radius R_e is simply the radius of the free rolling wheel [21]. According to definition of the slip ratio given by (2.5), a value for $S_r = -1$ means that the wheel is blocked. Inversely, when $S_r \geq 1$, the tire is spinning. The force for accelerating or braking the tire are therefore function of its slip ration. For slightly positive values of the slip ratio, they increase to reach a maximum. When this maximum is passed, they decrease steadily until the wheel starts spinning or get blocked. As there is not international agreement on the definition of the slip ratio, one use the following definitions [30],

$$\begin{aligned} S_b &= 1 - \left(\frac{v_x \cos\alpha}{\Omega R_e}\right), \\ S_t &= \left(\frac{\Omega R_e}{v_x \cos\alpha}\right) - 1, \end{aligned} \tag{2.6}$$

where S_b and S_t denote respectively the braking and tractive slip ratio. When $S_b = 1$ or $S_b = 0$, it means respectively that the wheel has blocked while braking or that the wheel is free rolling. When $S_t \leq -0{\cdot}5$, according to [30], it means simply that the wheel is spinning.

The maximum force that a tire is able to produce is proportional to the product between the normal force acting on the tire from the road and the friction coefficient at the interface between the tire and the road. Assuming that the friction coefficient is constant and independent of the load on the tire, the maximum resultant force F_r that a tire can produce is a bounded quantity,

$$F_r = \sqrt{F_x^2 + F_y^2} \leq \mu N, \tag{2.7}$$

where μ is the friction coefficient between the road and the tire and N denotes the normal force. It is important to highlight that μ is proportional to the load in reality. An important load gives rise to a higher friction coefficient and vice-versa [21]. For the simplicity, μ appears as a constant parameter in expression (2.7).

2.2.2 The lateral forces

The figure 2.5 shows the lateral force F_y as a function of the slip ratios S_b and S_t for different values of slip angles α_s. Notice first that for any slip ratio, the lateral force increases with the slip angle. The higher the slip angle, the smoother is the decrease of the lateral force with an increase or decrease of the slip angle. Second, notice that the maximum lateral force is reached for a braking slip ratio close to zero. In any case, increasing the slip ratio leads the lateral force to fall off rapidly.

The figure 2.6 and 2.7 show the lateral force as a function of the slip angle for several values of the slip ratio. Note that the lateral force tends to improve proportionally with the slip angle. This trend tends to fall off for a given slip angle, depending on the value of the slip ratio. For instance, when the slip ratio is close to zero, for both figures the maximum lateral force is approximately reached for $\alpha_s = 12$. As expected from the figure 2.5, if the slip ratio increase or decrease, from figure 2.6 and 2.7 one notices that the lateral force decreases proportionally.

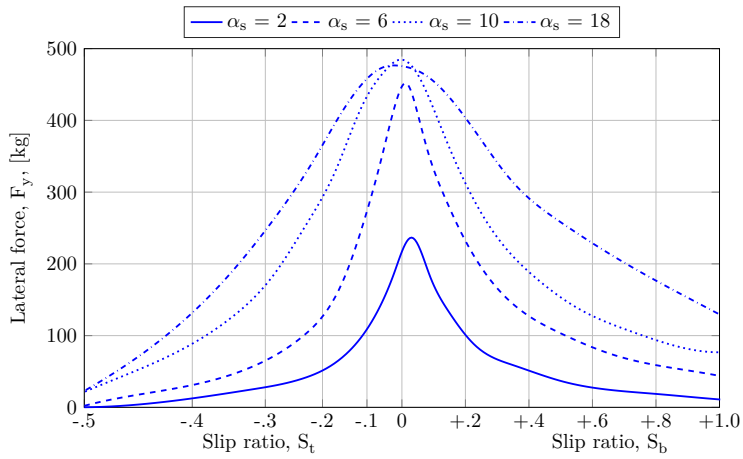


Figure 2.5: Lateral tire force as a function of the slip ratio for various slip angles [30]

The important aspects to remember here is that the lateral force F_y is very sensitive to changes on the slip angle and the slip ratio. Consequently, pushing the car to the boundaries of the $g-g$ diagram while cornering may be difficult to achieve if the model of does not take these aspects of the tire behaviour into account.

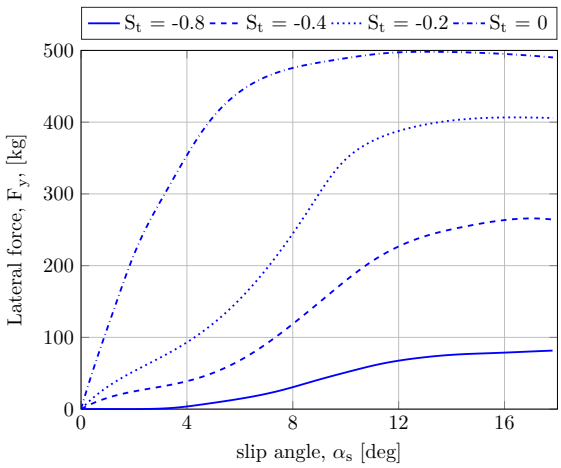


Figure 2.6: Lateral tire force F_y versus slip angle α_s for several tractive slip ratios S_t [30]

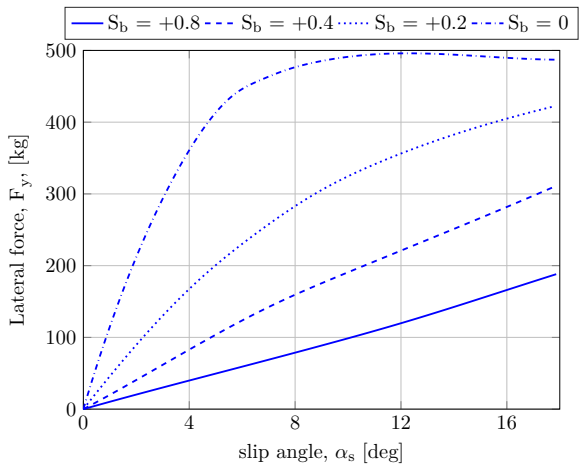


Figure 2.7: Lateral tire force F_y versus slip angle α_s for several braking slip ratios S_b [30]

2.2.3 The longitudinal forces

The figure 2.8 shows how the longitudinal force F_x varies with the slip ratio for several values of the slip angle α_s. According to [30], the positive and negative values on the ordinate axis denotes respectively the braking force and tractive force. The maximum braking and tractive

force occur for a slip ratio respectively slightly higher or lower than zero. Similarly to the lateral force, an increase in slip ratio causes the longitudinal force to decrease.

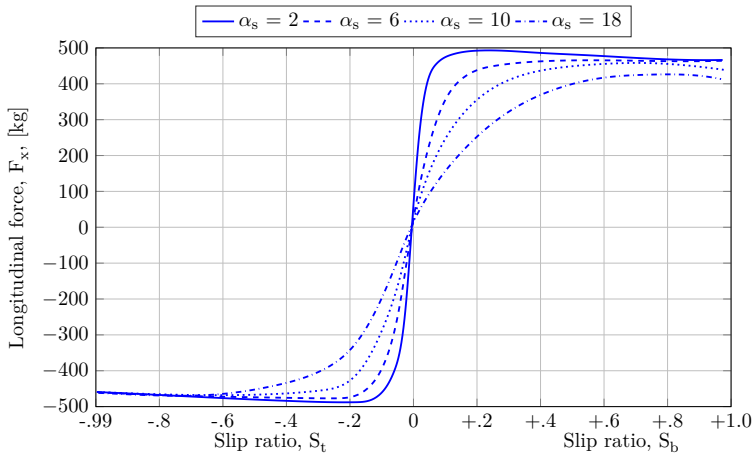


Figure 2.8: Longitudinal tire force F_x as a function of the slip ratio αs for various slip angles S_t and S_b [30]

It is here interesting to notice that the longitudinal force is also quite sensitive to the slip ratio and slip angle. Travelling on close to the boundaries of the $g - g$ diagram comes also together with the knowledge of the slip angle and the slip ratio. For instance, braking before a corner should occur with a very small slip angle to maximize the braking force. Even if this sounds simplistic, one shows below that some models cannot model these important aspects of the vehicle's dynamics.

2.2.4 The "g-g" diagram

For each tire of the vehicle, it is possible to draw a friction circle [21]. To do so, the lateral tire force F_y is reported on the abscissa axis of a graph and the longitudinal tire force F_x on the ordinate axis. The maximum radius of the friction circle is given by the normal force on tire multiplied by the friction coefficient. For the racing application, the tires of the vehicle are operated near the friction limit. In other words, the goal imposed by racing consists in reaching the maximum force for each tire. From the previous observations and depending on the situation, there is obviously a specific combination of slip angle and slip ratio which drives the tire close to this limit in an optimal way.

It is further possible the represent the vehicle friction circle. Indeed, as each tire has its own friction circle, it is possible to combine these to build the vehicle friction diagram by considering the vehicle as one single tire[21]. Normalizing the vehicle friction circle to acceleration units yields simply the "g-g" diagram. Therefore, the boundaries of the "g-g" diagram are defined by the maximum force each tire can produce. Which depends directly on the friction coefficient and the load on each tire of the vehicle. This is the reason why the "g-g" diagram provides a convenient way to evaluate the performance of a driver. Indeed, the forces on each tire of the car are difficult to measure and evaluate in practice. Inversely, the lateral and longitudinal acceleration can easily be measured. Reaching the boundaries of the "g-g" diagram consists in maximizing the available force on each tire of the vehicle. This is precisely what a skilled driver is able to accomplish and what the controller should be able to achieve.

2.3 System identification

As a reminder, the state vector z of the vehicle is given by,

$$z = \Big[x,\ y,\ \psi,\ v_x,\ v_y,\ \omega_z,\Big]^\top. \tag{2.8}$$

According to the aforementioned assumptions on the states of the car, the function f takes the following general form,

$$\dot{z} = f(z,u) = \begin{bmatrix} v_x \\ v_x \cos(\psi) + v_y \sin(\psi) \\ \beta v_x \\ f_{v_x}(z,u) \\ f_{v_y}(z,u) \\ f_{\omega}(z,u) \end{bmatrix}, \tag{2.9}$$

where $f_{v_x}(z,u)$, $f_{v_y}(z,u)$ and $f_{\omega_z}(z,u)$ represent the unknown dynamics of the vehicle that should be modelled. Note that the coefficient β accounts for the curvature of the road. The identification of $f_{v_x}(z,u)$, $f_{v_y}(z,u)$ and $f_{\omega_z}(z,u)$ is investigated in a general way and then compared to a more common model used for control [13], [27].

2.3.1 Generalized car dynamics

To obtain a generalized expression of the vehicle's accelerations, consider all the forces acting on the car in the horizontal plane as illustrated in figure 2.9. The forces acting at the front and rear tires of the vehicle are depicted along the x and y axis. The angles δ_l and δ_r are the individual orientations of the front wheels and depend on the steering δ. In the case of a traction mode race car, the driver accelerates the car by interacting with the environment through forces $F_{x_{fl}}$ and $F_{x_{fr}}$ on the front tires. In the case of propulsion mode car, the acceleration comes from the longitudinal force on the rear tires $F_{x_{rl}}$ and $F_{x_{rr}}$. The aerodynamic forces that the vehicle undergoes are expressed by the force F_d.

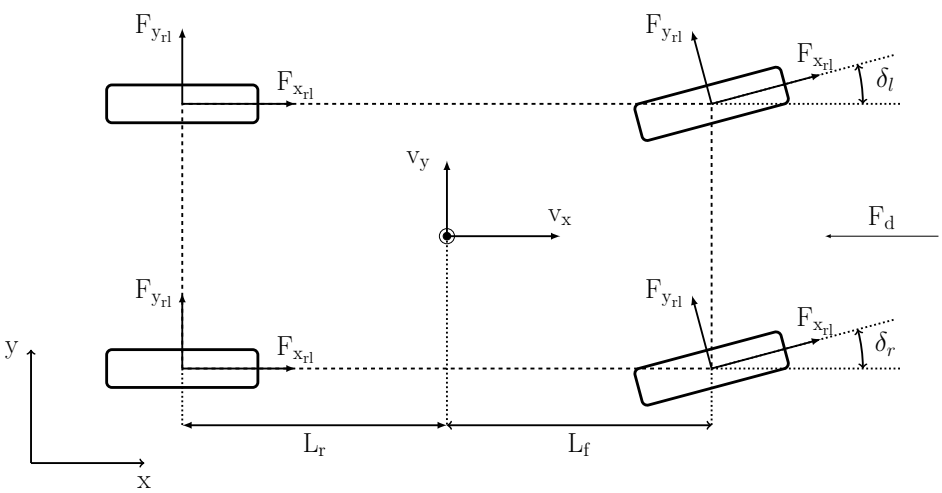


Figure 2.9: Illustration of the generalized car model in the horizontal plane

Given the mass m_v of the vehicle and its inertia along the z-axis expressed by J_z, applying Newton's second law yields an expressions of the car's accelerations along the x and y axis and the angular acceleration along the z axis,

$$
\begin{aligned}
m_v(\dot{v}_x - v_y\omega_z) &= \sum F_x, && (2.10)\\
m_v(\dot{v}_y + v_x\omega_z) &= \sum F_y, && (2.11)\\
J_z\dot{\omega}_z &= \sum M_z\cdot && (2.12)
\end{aligned}
$$

Equation (2.10) yields the time derivative of the longitudinal velocity,

$$
\dot{v}_x = v_y\omega_z + \frac{1}{m_v}\Big[F_{x_{fl}}\cos\delta_l + F_{x_{fr}}\cos\delta_r - F_{y_{fl}}\sin\delta_l - F_{y_{fr}}\sin\delta_r + F_{x_{rl}} + F_{x_{rr}} - F_d\Big]\cdot \tag{2.13}
$$

The acceleration or braking torque from the engine, or the brakes, is transmitted to the road through the longitudinal forces of the tires. However, depending on the slip ratio and the slip angle, the inputs may not have the expected effect as the maximum force the tire can produce is bounded. The consequence is that the wheels of the vehicle can start spinning or can get blocked. In other words, a poor knowledge on the behaviour of the tires can lead to situations in disagreement with the expectations of the driver. Notice also that the lateral forces acting on the front tires play an increasing role in the longitudinal dynamics as the steering increases.

Equation (2.11) yields the time derivative of the vehicle lateral velocity,

$$
\dot{v}_y = -v_x\omega_z + \frac{1}{m_v}\Big[F_{x_{fl}}\sin\delta_l + F_{x_{fr}}\sin\delta_r + F_{y_{fl}}\cos\delta_l + F_{y_{fr}}\cos\delta_r + F_{y_{rl}} + F_{y_{rr}}\Big]\cdot \tag{2.14}
$$

A good knowledge on the lateral dynamics is essential to improve the lap time in the corners. From (2.14), observe that for an increasing steering, the longitudinal forces on the front tires play in increasing role while the contribution of the lateral forces decreases. Which means that a complex combination of acceleration or braking and steering can lead to optimal trajectories on specific corners. The remarks here are the same as in (2.13). A good knowledge on the tire forces improves the understanding of the lateral dynamics and leads the manoeuvrability and stability of the car to improve in the corners.

Finally, equation 2.12 yields,

$$
\begin{aligned}
\frac{d}{dt}\omega_z = \frac{1}{J_z}\Big[L_f\Big\{ &- F_{x_{fl}}\sin\delta_l - F_{x_{fr}}\sin\delta_r + F_{y_{fl}}\cos\delta_l + F_{y_{fr}}\cos\delta_r\Big\}\\
&-L_r\Big\{F_{y_{rl}} + F_{y,rr}\Big\}\Big]\cdot
\end{aligned} \tag{2.15}
$$

The yaw-rate depends on the shape, dimensions and weight repartition of the car. The inertia of the car J_z tends to decrease the manoeuvrability of the car as it increases. The

center of mass of the car plays also a major role in the vehicle dynamics affecting the yaw rate dynamics but also the lateral and longitudinal dynamics of the vehicle. Its distance from the tires affect the yaw acceleration positively of negatively. The forces on the front tires tends to align the car with the corner while the forces on the rear tends to move away from it.

The expressions (2.13), (2.14) and (2.15) show that the dynamics of the car are complex even under simplified assumptions. Slip ratio and slip angles of the tires play a fundamental role in identifying the dynamics of the vehicle. However, many other parameters also affect the behaviour of the tires such as the speed of the vehicle, the temperature and pressure of the tires, the camber angle, etc. Furthermore, in practice, the roll and the pitch, which have been neglected so far, play a major role in the control and stability as they affect the effective load acting on each tire.

2.3.2 Modelling limitations

The generalized vehicle dynamic model presented in the previous section needs some modifications and further assumptions such that it can be used for Model Predictive Control applications. The forces acting on each individual tire are practically difficult to compute. A popular model discussed in [13] simplifies the car to a bicycle with one single steered front and rear wheel. The longitudinal, lateral and yaw dynamics of the bicycle are then given by,

$$\dot{v}_x = v_y\omega_z + a_x, \tag{2.16}$$

$$\dot{v}_y = -v_x\omega_z + \frac{2}{m_v}\Big[F_{z,f}\cos\delta + F_{z,r}\Big], \tag{2.17}$$

$$\dot{\omega}_z = \frac{2}{J_z}\Big[L_f F_f - L_r F_r\Big]. \tag{2.18}$$

The model of the tire is linear, and the resultant force on $F_{z,i}$ is expressed as, $F_z = -C_{\alpha_s}\alpha_s$. Where C_{α_s} is the cornering stiffness coefficient.

First, notice that the equation for the longitudinal dynamics in (2.16) neglects any force acting on the front or rear tires. The only contribution comes from the acceleration input a_x. Which means that the longitudinal dynamics are linearly affected by a_x. Whereas it is has been shown above that this is not the case in reality. Indeed, (2.16) takes into consideration neither the slip ratio nor the slip angle.

Second, the lateral dynamics as modelled in equation 2.17 take into account only the lateral forces acting on the front and rear tires. However, the generalized model has shown that they are also affected by the longitudinal forces acting on the front tires, depending on the steering. Nevertheless, the tire model here takes into account the slip angle to express the lateral forces. Note however that the linear tire model remains valid for small values of α_s only.

Third, similarly to equation (2.17), the yaw rate dynamics expressed in (2.18) neglect the longitudinal forces, which can potentially lead to a high mismatch for the cornering modes. Furthermore, some parameters as the inertia of the car along the z-axis and the position of the center of mass may be difficult to determine in practice.

Finally, it is clear that the bicycle model introduces a high mismatch between predictions and reality. Because of that, it is unlikely that the controller will be able to manoeuvre the car close to the boundaries of the "g-g" diagram and maximize the performance of the race car on the track.

2.3.3 Learning model

The modelling of the vehicular dynamics involves complex relationships and/or requires the estimation of many parameters such as coefficient of friction, inertia, and many others. Instead of trying to come up with a complicated model that takes into account every single parameter of the car, one can try to identify the model from collected data. The approach in this work is to learn the model $f(z, u, \theta)$. Where the parameters θ are adapted such that the longitudinal, lateral and yaw-rate dynamics match the recorded dynamics in an optimal way. Nonetheless, not all the model is to be learned, only the part dealing with the dynamics, i.e. the following equations,

$$\begin{aligned} \dot{v}_x &= f_{v_x}(z, u, \theta_{v_x}) \\ \dot{v}_y &= f_{v_y}(z, u, \theta_{v_y}) \\ \dot{\omega}_z &= f_{\omega_z}(z, u, \theta_{\omega_z}), \end{aligned} \tag{2.19}$$

where $\theta_{v_x}, \theta_{v_y}$ and θ_{ω_z} are the parameters of each sub-model.

For MPC applications, the continuous time equations in (2.19) can be discretized according to a first order Euler scheme. Consider that the state z evaluated a time t is written as $z(t)$. Consider the time step to be Δt. For notation simplicity, consider that $z(t)$ and $z(t+\Delta t)$ are respectively equivalent to z_k and z_{k+1}. Therefore, the Euler discretization of the f yields,

$$\begin{aligned} v_{x,k+1} &= v_{x,k} + f_{v_x}(z_k, u_k, \theta_{v_x}) \\ v_{y,k+1} &= v_{y,k} + f_{v_y}(z_k, u_k, \theta_{v_y}) \\ \omega_{z,k+1} &= \omega_{z,k} + f_{\omega_y}(z_k, u_k, \theta_{\omega_z}) \end{aligned} . \tag{2.20}$$

It has been shown in [13] that the discretization of continuous time model may impact the model in various ways. Large values of Δt makes it possible for the controller to look further into the future, while small values of Δt tend to produce better predictions. The choice of an optimal Δt is therefore the result of a trade-off between accuracy and future insight. The current approach also addresses this problem by learning directly to predict the future discrete states of the model. Therefore, instead of learning f_{v_x}, f_{v_y} and f_{ω_z}, one rather learns the models F_{v_x}, F_{v_y} and F_{ω_z},

$$\begin{aligned} v_{x,k+1} &= F_{v_x}(z_k, u_k, \theta_{v_x}) \\ v_{y,k+1} &= F_{v_y}(z_k, u_k, \theta_{v_y}) \\ \omega_{z,k+1} &= F_{\omega_w}(z_k, u_k, \theta_{\omega_z}) \end{aligned} . \tag{2.21}$$

Finally, the system identification process can be carried out based on collected data samples of the race car, namely the states z_k and inputs u_k of the system. The problem consists therefore in minimizing the squared error between the predictions and the observed true values of the system. The set of optimal parameters $\theta^\star$ is obtained as,

$$\theta^\star = \underset{\theta}{\operatorname{argmin}} \sum_{i=1}^{m} \frac{1}{m} \left(z_{k+1,i} - F(z_{k,i}, u_{k,i}, \theta)\right)^2 , \tag{2.22}$$

where i stands for the i-th data sample. The next chapters explains into more details how the problem 2.22 can be solved using two different approaches.

Chapter 3

System identification

This chapter presents the two methods used for system identification. First, we introduce the necessary notation. Second, the linear regression problem for the race car identification problem is presented together with the chosen features for the model. Third, it is discussed how it is possible to use fully connected neural networks to identify the dynamics of the car. Finally, one presents two different learning strategies to identify the model of the car in an off-line of on-line fashion.

3.1 Definitions

Regardless of the machine learning technique that is used, some common definitions are given here such as the feature batch and targets, performance evaluation, choice for regularization and the cost function being minimized.

3.1.1 Features and targets

The goal of the system identification is to find the best parameters $\theta^\star$ such that the model F can predict accurately the next states of the race car,

$$z_{k+1} = F(z_k, u_k, \theta), \tag{3.1}$$

where $F : \mathbb{R}^{n_z} \times \mathbb{R}^{n_u} \times \mathbb{R}^{n_\theta} \to \mathbb{R}^{n_z}$ contains the three models F_{v_x}, F_{v_y} and F_{ω_z} such that,

$$F = \begin{bmatrix} F_{v_x} \\ F_{v_y} \\ F_{\omega_z} \end{bmatrix} \tag{3.2}$$

These three sub-models are identified in the same way. Therefore, to simplify the notation, one use a more generic function h, denoted has the hypothesis function or the predictor in this work. It is simply defined as,

$$y = h(X, \theta), \tag{3.3}$$

where $h : \mathbb{R}^{m \times n_X} \times \mathbb{R}^{n_\theta} \to \mathbb{R}^m$, $y \in \mathbb{R}^m$ and $X \in \mathbb{R}^{m \times n_X}$ correspond respectively to the target and feature vector. They represent the data batch, or the data set, that contains all the required information to identify the dynamics of the vehicle.

The features are the quantities that characterize the model and the targets are the values that the model should be able to predict. Therefore, for each sample i of the data set, there exists a feature vector $X^{(i)} \in \mathbb{R}^{n_X}$ and a label or target vector $y^{(i)} \in \mathbb{R}$. In their simplest form, they are defined as,

$$X^{(i)} = \{z_k^\top, u_k^\top\} \quad \text{and} \quad y^{(i)} = z_{k+1,j}, \tag{3.4}$$

where $z_{k+1,j}$ represents that target state j for one of the three sub-models that must be identified. For this reason, three different target vectors y are built in practice. It is assumed in (3.4) that the number of features matches the number of states and inputs. However, for the case of linear regression for instance, many other features can be built from the initial feature vector. This is discussed into more details in the following section.

3.1.2 Data preprocessing

It is common in the field of machine learning to preprocess the features of the model. The most common preprocessing method consist in centring the features along the origin. For one type of feature j, the mean μ_j is computed over the whole batch as $\mathbb{E}(X_j)$. The result is subtracted for each sample i of the batch such that $\widehat{X}_j^{(i)} = X_j^{(i)} - \mu_j$, where $\widehat{X}_j^{(i)}$ is the centred result. An additional step scales the features with a unit standard deviation. This is carried out by computing the variance σ_j^2 of the features along each dimension j to obtain a centred and normalized set of features $\widehat{X}$. For each sample and each dimension one simply computes $\widehat{X}_j^{(i)}$ as,

$$\widehat{X}_j^{(i)} = \frac{X_j^{(i)} - \mu_j}{\sigma_j}. \tag{3.5}$$

In practice, the data preprocessing step helps gradient based methods, such as stochastic gradient descent [3], to converge faster. This is especially the case of neural network where the use of first order methods is very common.

3.1.3 Supervised learning problem

The cost function $\mathcal{L}$ associated with each i-th sample of the batch is defined as,

$$\mathcal{L}^{(i)}(X, \theta) = \frac{1}{2}\left(h(X^{(i)}, \theta) - y^{(i)}\right)^2. \tag{3.6}$$

The performance J of the model over a data set of size m is computed as the averaged sum of the cost associated to each sample of batch,

$$J(X, \theta) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}^{(i)}(X, \theta). \tag{3.7}$$

Therefore, the optimal set of parameters that minimizes J is denoted by $\theta^\star$ such that,

$$\theta^\star = \underset{\theta}{\operatorname{argmin}} \;\; J(X, \theta) \cdot \tag{3.8}$$

In the field of machine learning solving (3.8) can be referred to as a supervised learning problem, where the expected predictions of the model are explicitly given. However, there is a risk that the model obtained by solving (3.8) leads to very good results on the training batch and performs poorly on another data set. This problem is referred to as the over-fitting problem. This is the reason why the performance of the model is often evaluated on a test set other than the one used for the training procedure.

3.1.4 Regularization

A common way to prevent the model from over-fitting the training set consists in penalizing the parameters θ in the cost function such that they remain small. The two most popular techniques are the L_2 norm regularization and the L_1 norm regularization. The first one is used in the present work and this choice is motivated below. Therefore, the cost function $\mathcal{L}$ becomes,

$$\mathcal{L}^{(i)}(X, \theta) = \frac{1}{2}\left(h(X^{(i)}, \theta) - y^{(i)}\right)^2 + \frac{\lambda}{2}\|\theta\|_2^2, \tag{3.9}$$

where λ is the regularization weight that is adjusted depending on the problem.

The difference between the L_1 and L_2 norm regularization has been extensively investigated in the literature (e.g. [23]). The L_1 regularization method performs better than the L_2 method for problems that contain a high number of features, noisy features and/or sparse models for instance. It helps the model to select the features that have the highest influence on the predictions and discards the others. In the present work, the number of features is several order of magnitude smaller than the data point and therefore it is not necessary to use and L_1 penalization to do feature selection. However, we have decided to use the L_2 penalty to guarantee the the solution to our optimization problem is well defined.

3.1.5 Performance measure

The cost function J, evaluated for a regularization parameter $\lambda = 0$, gives a good insight on the individual performance for the predictions of a model on a given data set. Nevertheless, there are several other choices to evaluate the performance of the model [41], such as the Root Mean Square Error (RMSE) and the Mean Absolute Error (MAE). In the present work one decides to use the RMSE as a performance measurement. As a reminder, his definition is given by,

$$RMSE = \sqrt{\frac{\sum_{i=1}^{m}\left(y^{(i)} - h(X^{(i)}, \theta)\right)^2}{m}} \cdot \tag{3.10}$$

However, since the target values are different for each sub-model it is difficult to compare the predictions of the model collectively. This is the reason why ones defines also the Normalized Root Mean Square Error (NRSME),

$$NRMSE = \frac{RMSE}{\max(y) - \min(y)}, \tag{3.11}$$

where the RMSE is scaled with respect to the maximum and minimum target values to obtain a unit-less performance measurement.

3.2 Linear regression

In the case of linear regression, the function h is the result of the inner product between the features X of the model and its parameters θ,

$$h(X^{(i)}, \theta) = X^{(i)}\theta, \tag{3.12}$$

where $\theta \in \mathbb{R}^{n_\theta}$ and the batch sample $X^{(i)} \in \mathbb{R}^{n_X}$. Logically, the dimensions of X and θ satisfies $n_X = n_\theta$.

The advantage of using linear regression over other techniques such as neural network is that the system identification problem is formulated as a convex optimization problem. Very robust and computationally efficient techniques exist to solve such problems. In addition to that, many "out-of-the-shelf" solvers can be used to train the model, i.e. there is no need to build a specific solver. Furthermore, the number of parameters and features of the model remains generally low. As a consequence, the model can be evaluated at a very low computational cost, i.e. one inner product per data sample. This aspect is interesting for application where the computational time of the model is an issue. For the application of autonomous racing, although the present work does not really focus on this aspect, the optimal controller is in practice subject to time constraint.

On the other hand, choosing the appropriate features of the model may be a tricky procedure. For the present application, it is not reasonable fit a linear model of the car dynamics using the states and input of the model only. Indeed, this might be too restrictive has more complex dynamics may not be captured. The idea is to build a new feature vector Z by adding more features to the model which are eventually non linear. Based on the available knowledge of the vehicle's dynamics, it is possible to select wisely the additional features.

3.2.1 Choosing the features

For the car application, choosing the features of the model is the first step. In order to capture complex dynamics of the vehicle, it is possible to build non-linear features by manipulating the states and inputs. Indeed, it has been seen in the previous chapter that slip angles values and slip ratio depend on both the lateral and longitudinal velocity of the car for instance. Therefore, it may be interesting to try to capture these aspects by building the new feature vector Z.

For linear regression, a bias feature is added to the feature vector to take into account any constant aspect of the data to identify such that,

$$h(Z^{(i)}, \theta) = Z_0^{(i)}\theta_0 + Z_1^{(i)}\theta_1 + \ldots + Z_n^{(i)}\theta_n, \tag{3.13}$$

where the first feature $Z_0^{(i)} = 1$ for any sample of the batch. In order to determine the remaining features, one proceeds sequentially to come up with a feature vector common to each sub-model. Although one could argue that building an individual feature vector for each sub-model may possibly yield better results, the optimization procedure should discard

the less interesting features and take advantage of the ones which are meaningful to each sub-model.

Each model has to forecast its next state based on the knowledge of the current state and the estimated dynamics according to the definition of F given in the first chapter. It seems therefore important to consider that the first features should be longitudinal speed v_x, lateral speed v_y, and yaw rate ω_z. Following the same logic, it is further possible to assume a linear dependency between the inputs a_x and δ_f.

The longitudinal and lateral dynamics are respectively proportional to $v_y\omega_z$ and $v_x\omega_z$. Therefore, these two features are also added to Z. Note that these terms are not present in the dynamic equation of the yaw-rate. Nevertheless, it can be shown in practice that the predictions on the yaw-rate benefit from these two additional features.

It is not possible to come up with a future that has a direct relationship with the slip angle because, according to its definition, its computation requires the knowledge on the angular velocity of the tires. However, note here that a driver has not this knowledge either. In fact, the drivers can "feel" the slip ratio through the acceleration he gives to the car and the observed velocity of the vehicle. Therefore, one adds a last feature to the model a_x/v_x. If the tires of the car are slipping, this ratio should remain high. When the tires are blocked, they should be low. When the driver does not accelerate the car, this ratio is zero. Which corresponds to the definition of the slip ratio.

Finally, the feature vector Z is composed of ten different features,

$$Z = \Big\{1,\ v_x,\ v_y,\ \omega_z,\ a_x,\ \delta_f,\ v_x\omega_z,\ v_y\omega_z,\ \text{atan}(\frac{v_y}{v_x}),\ \frac{a_x}{v_x}\Big\}. \tag{3.14}$$

3.2.2 Training

Once the features are chosen, the objective of the system identification procedure is to solve the following unconstrained optimization problem to find the optimizer $\theta^\star$ of the model,

$$\theta^\star = \underset{\theta}{\text{argmin}}\ \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}^{(i)}(X,\theta), \tag{3.15}$$

or, according to the definition of $\mathcal{L}$ in (3.9) with the L_2 regularization term,

$$\theta^\star = \underset{\theta}{\text{argmin}}\ \frac{1}{2m}\sum_{i=1}^{m}\left(X^{(i)}\theta - y^{(i)}\right)^2, \cdot \tag{3.16}$$

The expression 3.16 can also be expressed in a matrix way,

$$\min_{\theta}\ \frac{1}{2m}\left(X\theta - y\right)^\top\left(X\theta - y\right) + \frac{\lambda}{2m}\theta^\top\theta \cdot\cdot \tag{3.17}$$

Since the problem to solve is convex, the minimizer $\theta^\star$ of problem (3.17) are the values of θ that cancel the gradient of the cost function,

$$\frac{\partial}{\partial\theta}\Big[\frac{1}{2m}\left(X^\top\theta - y\right)^\top\left(X^\top\theta - y\right) + \frac{\lambda}{2m}\theta^\top\theta\Big] = 0, \tag{3.18}$$

which are found by solving a linear system of equation,

$$\theta^\star = \left(X^\top X + \lambda I\right)^{-1} X^\top y. \tag{3.19}$$

The advantage of solving (3.19) is that no initial guess on the parameters is required. Whereas iterative methods such as gradient-based methods require one. However, note that these methods become more effective as the size of the batch increases or when more features are added to the model. Furthermore, for incremental learning applications, gradient-based methods become gain in interest as they can be warm-started with the previous optimal set of parameters to train a new better model.

3.3 Neural Networks

Neural networks have gain an increasing attention in the field of image and speech recognition over the past few years. Apart from being good classifiers they are also recognized to be good general learners and can therefore be use for system identification. For instance, they have been successfully used to identify the dynamics of the an helicopter in [25].

It has been shown in [5] that neural networks can learn any non-linear function. Compared to linear regression, the explicit definition of features characterizing a model is not required. Which makes it possible to identify a model in a more general way. It is therefore expected that a neural network can catch the non-linear dynamics of the race car.

The main drawback of using neural network is that the training process can be very sensitive to the initialization of the parameters. Which leads sometimes to the computationally expensive procedure of training several models to pick out the best one in the end. Furthermore, the number of parameters in a neural networks is usually large. Which slows down the prediction and learning rate. Therefore, when time is an issue, the architecture of the network and/or the learning algorithm should be chosen very carefully.

3.3.1 Definitions

Fully connected feedward neural networks are considered in this work. They are composed of one input layer, one output layer and one or several hidden layers. Each layer as one or several neurons. The number of neurons in the input layer equals the number of features of the model. The features are simply fed to the input layer, which propagates through all the neurons of the first hidden layer, etc. until the signal reaches the output layer. The result of a forward-pass yields the prediction of the model. Figure 3.1 illustrates the typical architecture of a neural network used in this work.

The input layer is composed of five neurons, in agreement with the number of features given in X. There are three hidden layers, each made of height neurons. The last layer contains a single neuron that plays an essential role. This is discussed below into more details. The number of neurons on each layer together with the number of hidden layers determines the architecture of the network. The more hidden layers, the deeper is the network, and the better are his abstraction capabilities.

The parameters of the network are defined by the weights on the connexions. The distinction is done between the weights and biases. The weights are associated to each visible edge of the network, as illustrated in figure 3.1. Inversely, the biases are generally kept invisible. Similarly to linear regression where a unit feature is added to the model, the bias corresponds

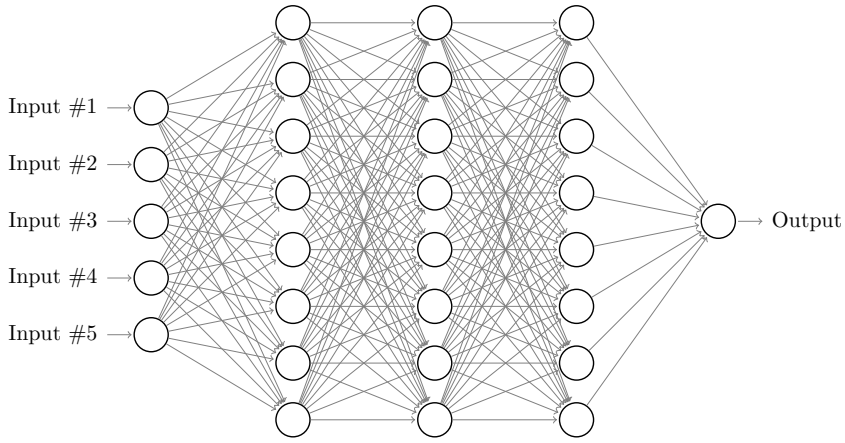


Figure 3.1: Illustration of a fully connected neural network

to the weight on the connexion between the corresponding neuron and a unit input. This is illustrated in figure 3.2.

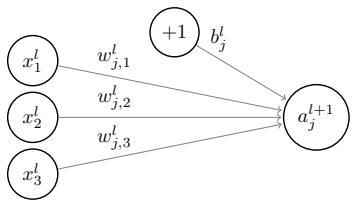


Figure 3.2: Illustration of the bias unit in neural networks

To compute a forward pass, consider again figure 3.2. From layer l, one computes the value of neuron j in the next layer $l+1$. This is carried out by summing up each neuron value x_k^l multiplied by the appropriate weight value w_{jk}^l and the bias b_j^l,

$$a_j^{l+1} = \sigma(\sum_{k=1}^{n_k} w_{jk}^l x_k^l + b_j^l), \tag{3.20}$$

where n_k is the number of neurons in layer l and the function σ is the non-linearity function associated to each neuron. It is also called the activation function or the squashing function. The choice for the squashing function used in the present work is motivated below. A forward-pass on the neural network is carried out by applying equation (3.20) sequentially for each neuron from the input layer to the output layer.

3.3.2 Reconstruction layer

In order to use the neural network for system identification, the single neuron in the output layer must have a linear squashing function. The last layer of the network can be seen as a reconstruction layer [25]. This is illustrated on the figure 3.3 where the distinction is made between the hidden units of the network and the reconstruction layer.

Consider that the weights and biases in the hidden units are denoted by w_h and b_h, the weights and bias in the reconstruction layer are denoted by w_r and b_r. The result of the network can be expressed as,

$$h(X^(i), \theta) = w_r^\top \sigma(w_h^\top X + b_h) + b_r, \tag{3.21}$$

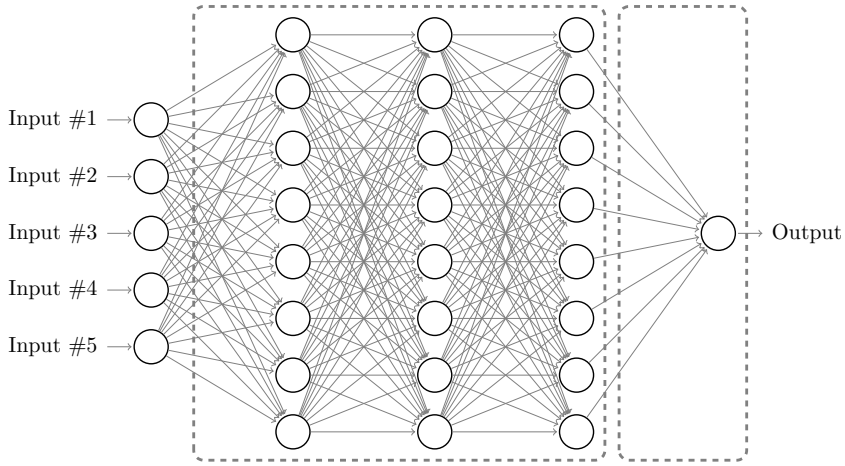


Figure 3.3: Illustration of the hidden layers and reconstruction layer

where the parameter vector θ is simply built from the concatenation of all the biases and weights of the network.

Note that equation 3.21 remains valid only for a network composed of one single hidden layer. While it becomes false for several hidden layers, it is its interpretation that matters. Indeed, the hidden units can be interpreter as a tool that builds, from the initial feature vector X, many others different and non-linear features. These are then provided to the reconstruction layer. Similarly to linear regression, the output of the network is then a linear combination of the new and more complex features built by the hidden units. This interpretation of the network gives also a clear understanding on the role of the hidden units in the network. In other words, adding more hidden units to the network leads to an increasing number of features of the model. However, a model that contains too many features is likely to over-fit the data set. Therefore, the number of hidden units should be balanced to find the right trade-off.

3.3.3 Squashing function

In the present work, the investigated squashing function for the system identification are the logistic function $1/(1 + \exp(-x))$, the hyperbolic tangent $\tanh(x)$ and the rectified linear unit function (ReLU) ReLU(x) = max(0, x). These are illustrated in figure 3.4. Note that the leaky ReLU function is also depicted on the figure, its use is motivated below.

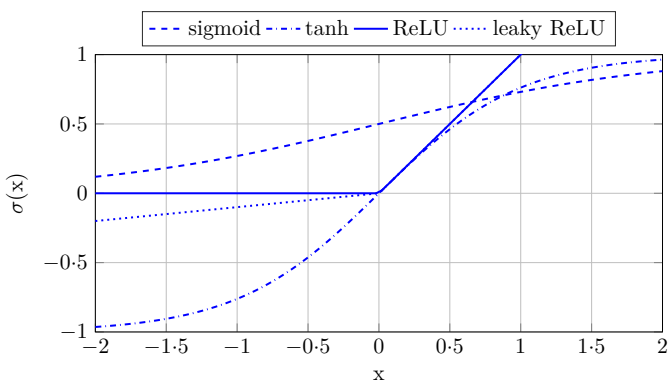


Figure 3.4: Illustration of the squashing functions

The logistic function is bounded between 0 and 1. However, the use of the antisymmetric hyperbolic tangent function is usually preferred. It promotes the convergence properties of the training algorithm according to LeCun et al. in [16] and [15]. However, both the logistic and the hyperbolic tangent activation functions suffer from the vanishing gradient problem. Indeed, the parameters of the network should all be initialized near the origin such that the activated values are neither too large nor too big. In both cases, evaluating the gradient of the activation function for a large or small value yields a gradient close to zero, which slows down the training procedure.

The use of ReLU units has two benefits. First, they are cheap to compute as they do not involve the exponential function. This aspect fastens the learning process significantly. Second, as stated in the work of Glorot et al. [9], they are linear by part and the vanishing gradient problem is therefore in-existent. Furthermore, stochastic gradient descent using ReLU functions has been shown to converge faster than with the logistic or hyberbollic tangent function in the work of Krizhevsky et al. [14].

Networks made of ReLU units sometimes suffer from the dying ReLU problem. This happens when the values of the unit are located on the negative domain. Then, the resulting gradient remains nought for the rest of the training procedure. This can be caused during the initialization. Or during the training when the gradient steps are too high. Therefore, Leaky ReLU were introduce to tackle this specific problem (e.g. [19], [42]). They are defined as $\sigma(x) = \max(ax, x)$ where $0 < a < 1$ is a constant value. According to their definition, Leaky ReLUs have always a non-zero gradient and can then recover from the negative domain.

3.3.4 Gradient descent optimization methods

The different optimization methods used in this project are described here. This work has focused on first order methods only. While second order methods provide better convergence properties [20], the first order methods have the benefit of being easily implemented and computationally efficient.

Since the size of the neural networks used in the present work remain small in comparison to other deep learning applications, it would be justified to use second order methods. However, while computing the gradient of a neural network can be done in a very efficient way using the back-propagation algorithm [15], computing the Hessian of a neural network is not straightforward. Further work will investigate on the use a the free Hessian method proposed by J. Martens in [20]. Which may be a promising way to obtain better results.

Stochastic gradient descent

The objective of the learning algorithm is to obtain the optimal set of parameters $\theta^\star$ such that the cost function over the training batch is minimized,

$$\theta^\star = \underset{\theta}{\operatorname{argmin}} \ \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}^{(i)}(\theta) \cdot \tag{3.22}$$

In the same way as for the linear regression, a L_2 regularization term is added to the cost function to prevent over-fitting of the training samples.

$$\mathcal{L}^{(i)}(\theta) = \frac{1}{2} \left(h(X^{(i)}, \theta) - y^{(i)} \right)^2 + \lambda \|\theta\|^2, \tag{3.23}$$

where λ is the regularization parameter.

It is important to note that since the neural networks are made from non-linearities, the function $h(X^{(i)}, \theta)$ in the cost function is non-linear. It results that problem (3.22) is a non-convex. Consequently, using gradient descent method does not guarantee that θ_{star} is a global minimum. For this reason, the initialization of the parameters θ_0 affects significantly the quality of the solution.

The computational cost to compute the gradients of the cost function with respect to the parameters of the network grows linearly with the size of the training batch. As it is common to train a neural network over a large set of data, it is convenient to obtain an estimate of the gradient over the whole batch using only a few sample from this batch. This is the main idea behind the stochastic gradient method. It is assumed that all the samples from the batch are drawn from a random distribution. Therefore, computing the expected value of the gradient on a subset of the batch yields an approximation of the gradient. This subset is often referred to as the mini-batch.

Consider the batch B of size m, and a mini-batch S of size n, such that $n < m$ and that $S \subset B$, the gradient of (3.23) is computed as,

$$\nabla J(\theta) = \sum_{i \in S} \frac{1}{n} \frac{\partial}{\partial \theta} \mathcal{L}^{(i)}(\theta) \cdot \tag{3.24}$$

In its simplest form, the Stochastic gradient descent algorithm takes the form of,

$$\theta_{k+1} = \theta_k - \eta \nabla J(\theta), \tag{3.25}$$

where η is a user defined learning rate.

In this work, other methods were implemented. Because (3.25) may converge very slowly or not converge at all, depending on the tuning of the parameter η.

The momentum method

The stochastic gradient descent algorithm in (3.25) has difficulties to cope with situations where the curvature of the cost function becomes very small or when the curvature along one dimension exhibits very different values along other dimensions [40]. In the first case, the step size η should be increased to take longer steps. In the second case, the step size over each dimension should be adapted such that the method smoothly finds its way towards the minimum. Instead, it will keep oscillating in the vicinity of the optimum because it has no knowledge on the surrounding curvature. The momentum method tries to address this problem ([26], [29]),

$$\begin{array}{rcl} v_{k+1} & = & \gamma v_k + \eta \nabla J(\theta) \\ \theta_{k+1} & = & \theta_k - v_k \end{array}, \tag{3.26}$$

where γ is referred to as the momentum term.

The term v accumulates the gradients of the function over the past iterations depending on the value of γ. Which is usually set to 0·9. As many steps are performed, the gradients keep accumulating. If the gradients along one dimension have not change of sign over the past iterations, the momentum will cause higher steps to be taken along the same dimension.

Whereas they cancel each other if the sign of the gradients along one dimension changes often. Consequently, the momentum acts as a damper to prevent too strong oscillations in areas where a high curvature can cause the standard method to oscillate. It has therefore an improved convergence rate and is less prone to oscillations.

Nesterov accelerated gradient method

The momentum method is sometimes too aggressive in term of step size. It has therefore difficulties to slow down in the vicinity of a minimum. Where a smaller step size is required. The Nesterov Accelerated Gradient method (NAG), described in [22] and [29], uses the momentum term is a slightly smarter way. It anticipates on the next move of the momentum update. The NAG algorithm performs the following updates,

$$\begin{array}{rcl} v_{k+1} & = & \gamma v_k + \eta \nabla J(\theta - \gamma v_k) \\ \theta_{k+1} & = & \theta_k - v_k \end{array} . \tag{3.27}$$

Instead of computing the gradient for the current position, the gradient is computed for the approximated next position. Which is roughly approximated as $\theta_k - \gamma v_k$. The NAG method anticipates the move of the momentum term to adapt the gradient descent step accordingly. In other words, it consists in letting the momentum do a jump in the direction of the accumulated gradients, observe the result and apply a correction to find a better position. In practice, the NAG method outperforms often the momentum method.

RMSprop

The definition of the step-size η is the same for each parameter of the network for both the momentum and NAG methods. The disadvantage of this approach is that the magnitude of the gradients may be different from one parameter to another. It can therefore be convenient to tune the step size η along each dimension. Similarly to second order methods that automatically adapt the step size given the Hessian matrix. The RMSprop method introduced by G. Hinton in [11] tries to address this problem,

$$\begin{array}{rcl} \delta_{k+1} & = & \gamma \delta_k + (1-\gamma) \nabla J(\theta)^2 \\ \theta_{k+1} & = & \theta_k - \frac{\eta}{\sqrt{(\delta_{k+1}+\epsilon)}} \nabla J(\theta) \end{array} . \tag{3.28}$$

The algorithm collects the square of the gradients with respect to each parameters for each new iterate. They are averaged according to a parameter γ. Which is usually set to 0·9. If the collected gradients along one dimensions are large, the step size along the same direction is decreased. In order to prevent too large steps being taken. Inversely, when the gradients are small along one dimensions, the step size is increased to enable larger steps and increase the convergence rate.

3.3.5 Parameter initialization

The initialization of the neural network is carried out before the gradient-based method starts. The way they are initialized affects significantly the quality of the solution. For linear regression for instance, a common way to initialize the weights is to set them to all to zero. Since the problem is convex, the initialization does not impact the quality of the final solution. This approach becomes inconsistent for neural networks. Initializing all the

weights to zero prevents the network from learning anything. Because all the neurons will see the same signal and therefore the same gradients. In practice, the weights are initialized with small random numbers and the biases are initially set to zero.

In the present work, two initialization methods have been implemented. The first one is a method introduced by Glorot et al. in [8]. The method consist in initializing the weights w_{jk} on each layer from a uniform distribution,

$$w_{jk} \backsim \mathcal{U}(\frac{2}{n_k + n_j}, \frac{2}{n_k + n_j}), \tag{3.29}$$

where n_k and n_j denote respectively the number of units int the previous layer and in the next layer. Initializing the weights in this way enables the weights to be scaled in such a way that the signals through the network can propagate easily from the inputs to the outputs.

The second initialization is proper to ReLU units. Is introduced in the work of He et al. in [10]. It consist in initializing the weights according to a Gaussian distribution with zero mean and a standard deviation inversely proportional to the number of units from the previous layer,

$$w_{jk} \backsim \mathcal{N}(0, \frac{2}{n_k})\cdot \tag{3.30}$$

3.4 Learning strategy

One presents here two different learning strategies. The non-incremental learning approach is the most conventional way to identify a model. The learning is carried out using a large data set whose samples have been collected a priori. Once the learning has been performed, the model stops learning. By opposition, incremental learning continuously retrain the model when new data become available. For the race car application, the incremental learning approach is attractive for several reasons. First, the initial set of data may not describe extensively the dynamics of the car. Second, an adaptive model can automatically adapt on the road conditions.

3.4.1 Semi-incremental learning

The non-incremental learning approach is useful when an extensive number of data samples are available beforehand. However, pushing the car to its highest performances requires very specific data. The only way to obtain these data would be to let a skilled driver drive on the specific track. The recorded data are then used to train the model. This approach is not realistic in practice for at least two reasons. First, the recorded data may potentially not describe the dynamics of the car extensively. Second, for the purpose of autonomous racing, the car may not even have been designed for a human. Therefore, learning once and for all a fixed model and provide it to the controller is considered to be a too restrictive learning strategy in the present work.

It is possible to take advantage of the situation to perform the training of the model in a smarter way. Indeed, the race car collects new data continuously. Among them, some may be representative of previously unknown dynamics. Before completing the next lap, it is possible to stop the car, identify a new better model, and start the next lap. Ideally, this semi-incremental learning strategy is carried out after each lap. The advantage of this

approach is that, the training time is not an issue. In the case of training a neural network for instance, several models can be trained and cross-validated to pick up the best one before the start of the next lap.

Finally, it can be argued that the need to stop the car from one lap to another is not realistic during a competition. This problem can easily be addressed by training the model asynchronously or in parallel of the controller. Such that the controller will start to use the new model of each new lap when it becomes available. Note also that this approach requires an initial model, which can be identified from some a priori available data.

3.4.2 Incremental learning

A definition of incremental learning is given by Giraud in [7]. The incremental learning algorithm trains the model over and over again as long as new data samples become available. For the race-car application, a new data samples is provided for each time step. The model of the race car automatically adapts it-self when the environment changes, if it starts raining suddenly for instance. Note that, depending on the size of the track and the required time to complete the track, this remain also true for the semi-incremental learning strategy.

For the present application, the incremental learning approach can be limited due to time issue. Indeed, it may be practically impossible to achieve the training of a new model and solving the optimal control problem for each time step. This problem can be address by using iterative learning methods. Indeed, they can always be warm-started with the previous parameters of the model. A very small number of iterations are then performed to come-up with a slightly better model.

For this configuration, the management of the training batch is critical. Indeed, there is a risk that the model forgets previously known characteristic of the vehicle dynamics. For this reason, the new collected data are added to a training batch. Which contains the previously observed data. Whenever a new sample is collected, the last one is discarded. When the size is large, the learning algorithm performs enough iterations on each sample to learn the associated dynamics. In the long-run, it is expected that the model will always improve. Note that this strategy may be especially efficient when the model is identified with neural networks.

Finally, these two learning strategies solve the problems related to a unique attempt of identifying the model. Indeed, the adaptive model will always take into consideration new collected data and modify its knowledge about the environment accordingly. However, it is possible that the collected data do not lead to improvements of the model. And there is a risk that is repeated over and over again. In fact this question is related to the intrinsic quality of the collected data samples. If the new data samples are not interesting enough, the model will not improve. Similarly to a curious driver that always wants to improve its performances on the track, it is possible to take advantage from the incremental learning strategy to make the race car "curious". In such a way, the car may collect better data samples to learn a better model which leads consequently to better control performances. This is discussed in the next chapter.

Chapter 4

Curiosity

This chapter shows how it is possible to take advantage of the incremental learning strategy to actively improve the model of the race car. Indeed, the system identification process is generally carried out in a passive way. In other words, there is no communication between the controller and the system identification module to actively improve the model. This field of research is referred to as active learning [37] and consists in using the controller to guide the system such that the system identification process is provided with more "interesting" data to acquire a better knowledge of the race car dynamics.

Nevertheless, it is not an easy and straightforward task to determine weather or not a set of data is interesting. To develop an active learning algorithm, the present work is essentially based on the theory of creativity, fun and curiosity developed by Schmidhuber in [33], [32], [31], [36], [34] and [35]. The main idea of the theory states that an event is interesting if it contains some useful information that leads the model to improve. Whereas, event which are not likely to improvement the performance of the model should be discarded.

4.1 Theory of curiosity

A system or agent that is equipped with a controller, an adaptive model and an appropriate learning algorithm has the ability to become a curios agent. The controller used in the present work is the Learning Model Predictive Controller. Note that the next chapter discusses the controller into more details. The adaptive model has been discussed in the previous chapter of this work. Note that the word adaptive implies logically that one uses the incremental learning strategy. The learning algorithm is the optimization algorithm that trains the model. For instance, a neural network model can be trained using stochastic gradient descent. The ultimate step consist in being able to generate the curiosity cost. Therefore, one defines first the concept behind the curiosity. Then, one shows how these four elements can be put together to obtain a curios race car.

4.1.1 Intrinsic motivation and external goal

Consider a system or agent that has the ability to continually improve or adapt its model. The goal of the system is to improve its performance over time at achieving a given task. The task of interest here is namely to improve the time performance of the race car on the circuit. For each time t, the agent can measure its states $z(t)$ and respond with a set of

input $u(t)$. The way the agent selects $u(t)$ is dictated by the controller which maximizes a cost function $c(t)$ [34],

$$c(t) = \mathbb{E}\left[\sum_{\tau=t+1}^{\infty} l(\tau) \,\middle|\, h(\leq t) \right], \quad \forall t \in [1, T[\tag{4.1}$$

where $l(t)$ measures the cost of completing the given task for each t. Note that $c(t)$ can be evaluated over a finite or infinite horizon. One keeps here the infinite horizon notation. The past states and input are recorded in the history vector h, where $h(t)$ is made of $z(t)$ and $u(t)$.

As the agent travels through the environment, it tries to maximize $c(t)$. Meanwhile, the collected history samples are used to improve the adaptive model. In case the agent focuses only on maximizing the performance of its assigned task, the predictor will eventually improve given some recorded trajectories. It may happen, at some point, that no further improvements of the model can occur, leading consequently the agent to its maximum performances. However, if the agent is intrinsically motivated [34] to improve its predictor or adaptive model. Then it may eventually lead to better final performances because the agent can learn previously unknown aspects of its environment. This behaviour can be referred to as curiosity, creativity or fun [35].

To enable the curious behaviour of the agent, the distinction is made between the external cost and the intrinsic cost,

$$l(t) = q(t) + r(t), \tag{4.2}$$

where the external cost $q(t)$ expresses the will to achieve the given task with a certain degree of performance while the intrinsic cost $r(t)$ motivates the agent to improve itself. Maximizing $c(t)$ express therefore the trade-off between exploiting and improving the model. The first idea here is that maximizing the intrinsic cost $r(t)$ can ultimately lead to a better external cost $q(t)$. The second idea is that the intrinsic motivation may eventually ease the agent at maximizing $q(t)$, i.e. find a quicker way to achieve a better performance.

4.1.2 General the intrinsic cost

For each $t \in [1, +\infty[$, there exists an history $h(\leq t)$, a world predictor $p_M(t)$ and a curiosity predictor $p_R(t)$. The history $h(t)$ is defined as the concatenation of the states and inputs at time t, i.e. $h(t) = \{z(t), u(t)\}$. A predictor $p(t)$ is defined by its parameters at time t. Note that to be consistent with the notation, one should express $p(t)$ according to its parameters θ defined at time t. However to ease the notation and to follow the terminology used by Schmidhuber in [34], one prefers to stick to $p(t)$. To generate the intrinsic cost agent $r(\leq t)$ and train the curiosity predictor p_R one repeats for each t:

1. The controller use the current history $h(t)$ to select and execute the input $u(t)$.
2. The next state $z(t+1)$ is recorded and the history $h(\leq t)$ is updated with $u(t)$ and $z(t+1)$ to obtain $h(\leq t+1)$.
3. The performance of the predictor $p_M(t)$ is evaluated on $h(\leq t+1)$ by performing,

$$C\left(p_M(t), h(\leq t+1)\right) \cdot$$

4. The learning algorithm trains $p_M(t)$ using $h(\leq t+1)$ to obtain a better predictor $p_M(t+1)$.
5. The performance of the predictor $p_M(t)$ is evaluated on $h(\leq t+1)$ by performing,

$$C\left(p_M(t+1), h(\leq t+1)\right) \cdot$$

6. The curiosity reward $r(\leq t+1)$ is updated,

$$r(\leq t+1) = g\Big[C(p_M(t), h(\leq t+1)), C(p_M(t+1), h(\leq t+1))\Big] \cdot$$

7. The training algorithm uses $r(\leq t+1)$ to train a better curiosity predictor $p_R(t+1)$.
8. Provide $p_M(t+1)$ and $p_R(t+1)$ to the controller.

This scheme is highly inspired from [34] and keeps almost the same notation rules for the sake of simplicity. Some modifications however have been brought such that it can be used with a Model Predictive Controller. Indeed, the scheme as presented in [34] is suitable for Reinforcement Learning. Furthermore, Here, it is important to point out that the performance of the model is measured according to a function C, which is an essential tool in computing $r(\leq t)$. It is discussed below how to choose C and the function g to generate $r(\leq t)$.

The main modification brought to this scheme, comes from an idea suggested by the same author in [31]. The idea consists in being able to predict the future intrinsic cost based on the passed observations. For this purpose, an additional predictor is trained and used by the controller. Note that is uses the same inputs as the adaptive model. This will be further discussed in the last section of this chapter.

4.2 Performance of the adaptive model

Measuring the quality of the adaptive model is one of the key aspect towards generating an intrinsic or curiosity cost. However, the question of how measuring it has not been answered so far. There exist several ways to measure the quality of the model. Schmidhuber defines in [34] and [35] different ways of evaluating the ability of the model at producing accurate predictions. The present work focuses on the ability of the model to predict the next states only.

4.2.1 Performance evaluator

A naive way [34] of measuring the quality of the predictor would be to compute the square of the errors between the observed values and the predictions at time t. This is done as,

$$C(p_M(t), h(\leq t)) = \|\zeta(p_M(t), h(t)) - s(t)\|^2, \tag{4.3}$$

where $C(p_M(t), h(\leq t)) \in \mathbb{R}$, p_M is the world predictor and the result of $\zeta(p, h)$ yields the predictions of p given the past state input sequence described by h. Here, only the most recent event is taken into account and past events do not determine the performance of the model. The performance misses therefore critical information that happened in the past and that may potentially be useful towards improving the model.

Logically, one comes with another performance measurement that takes into account all the past events that were recorded in the history. The performance becomes the expression of the sum of all the squared error for each time t of the history,

$$C(p_M(t), h(\leq t)) = \sum_{\tau=1}^{t} \|\zeta(p_M(t), h(\tau)) - z(\tau)\|^2, \tag{4.4}$$

where $C(p_M(t), h(\leq t)) \in \mathbb{R}$. The result of expression (4.4) may be difficult to compute for two reasons. First, depending on the size of the history, it might be practically impossible to compute within a reasonable amount of time. Second, it may be difficult to store all the past samples of the history.

The contribution of the present work is to limit the size of the history h to a certain maximum value n_h and to adapt the performance evaluation of the model given in [34] in the following way,

$$C(p_M(t), h(\tau)) = \|\zeta(p_M(t), h(\tau)) - z(\tau)\|^2, \tag{4.5}$$

where $\tau \in [1, t]$, $C(p_M(t), h(\tau)) \in \mathbb{R}$ and $C(p_M(t), h(\leq t)) \in \mathbb{R}^{n_h}$. The advantage of such a performance evaluator is that for each sample of the history $h(\tau)$, there exists an associated model quality measurement C. Since the expression of the intrinsic cost is based on this same expression of the model performance, one expects that it will be easier to point out which states require more attention than others.

In the present work, the squared error is used to measure the error between predictions and true values but other choices are possible. For instance the euclidean distance, absolute value or the Kulkback-Liebler [39] in the case the predictions and true values can be drawn from probability distributions.

4.2.2 Performance of the model

The dynamics of the race car are modelled with three distinct sub-models. It is suggested in [31] to measure the global model performances or the local model performances. As mentioned above, it is assumed that p_M contains the three sub-models of the race car dynamics such that it contains the appropriate parameters of each sub-models. One introduces now $p_{M,i}$, the i-th sub-model of p_M. In other words, $p_{M,i}$ is the predictor over the states z_i. Therefore, the performance measurement as given in (4.5) can also be expressed as,

$$C(p_M(t), h(\tau)) = \sum_{i=1} \left(\zeta(p_{M,i}, h(\tau)) - z_i(\tau)\right)^2, \tag{4.6}$$

where C becomes then the expression of the global model performance at time τ.

The local prediction error can simply be computed as,

$$C(p_{M,i}(t), h(\tau)) = \left(\zeta(p_M(t), h(\tau)) - z(\tau)\right)^2, \tag{4.7}$$

where $C_{t,i}$ accounts for the performance of the local performance of the i-th submodel.

This work focuses on the the global performance of the model. The advantage of this performance measure is that there is a single performance measurement only. Consequently,

a single intrinsic cost is taken into consideration for each time t. Dealing with one single curiosity cost ease the procedure of forecasting future curiosity costs. Indeed, training several curiosity predictors may become very expensive in terms of computational time. Notice, however, that a performance measurement per sub-model would make is possible to improve the performance of each sub-model individually. Which may potentially lead to better results. But this requires further investigations.

4.3 Curiosity cost

The curiosity can be defined in several ways. One focuses in the present work essentially on its definition given in [33], [32] and [31]. Note however that other definitions are given in [39] and [12].

The curiosity term or intrinsic cost r for each time t of the history $h(\leq t)$ can be computed in a general way as,

$$r(\leq t) = g\Big(C\Big(p_M(t-1), h(\leq t)\Big), C\Big(p_M(t), h(\leq t)\Big)\Big) \tag{4.8}$$

where C has been discussed above and $g(\cdot,\cdot$ is to be defined. Note that $r(t) \in mathbbR$ denotes the intrinsic cost for time t and that $r(\leq t) \in \mathbb{R}^{n_h}$ denotes the intrinsic cost for every sample of the history. Three definitions of the curiosity cost r can be obtained by playing essentially on the definitions of g and C.

4.3.1 Curiosity as the model error

Whenever there is a mismatch between the reality and the predicted values of the adaptive-world model, it means that there is an opportunity to learn something out of the environment. Such that the learning algorithm will improve the model as stated in [33] and [32]. The curiosity cost is then simply proportional to the model mismatch. As a consequence, the performance evaluator C keeps its initial definition and the function g is defined as $g(a, b) = b$. Therefore, the expression (4.8) becomes,

$$r(\leq t) = C\Big(p_M(t), h(\leq t)\Big)\cdot \tag{4.9}$$

Computing $r(\leq t)$ is such a way assumes implicitly, and optimistically [35], that a mismatch leads necessarily to an improvement of the model. This point of view is limited due to at least two different observations. First, there is a risk that it leads the agent, or the car in the present case, to parts of the environment where nothing at all can be learned because predictions errors will remain high due to noise for instance. Second, it may lead the race car to infeasible situations, i.e. crashing in the worst case. Remember here that the definition of interestingness is related to the ability to discover new events that can be understood and that lead to improvements of the model. Nevertheless, the present definition of r remains interesting to implement together with a MPC controller as the controller should prevent the system from going infeasible.

4.3.2 Curiosity as improvement of the model

To tackle the aforementioned problem, the idea here is to generate a curiosity cost that is proportional to improvements of the model. In other words, the agent should be motivated to go towards parts of input-state space where there is a high expectation for model improvement. First, the agent should not spend time in parts of the environment which are already well known. Second, no training should be spend in parts of the environment where one expects no improvement at all. In the present case one simply has $g(a, b) = a - b$. The intrinsic cost can then be expressed as,

$$r(\leq t) = C\Big(p_M(t-1), h(\leq t)\Big) - C\Big(p_M(t), h(\leq t)s\Big)\cdot \tag{4.10}$$

Maximizing $c(t)$ encourage the system to improve the model in an optimal way. To be more specific, the first term on the right side of (4.10) should always be higher than the second one due to the definition of C and assuming that the learning algorithm always improves the model. Therefore, maximizing $r(t)$ leads to trajectories where the difference in model performances is high before and after the learning algorithm has improved p_M. If $r(t)$ is high, it means that p_M has improved from $t-1$ to t. Whereas, if the difference is small, it means that no improvement occurred and those trajectories are discarded. Maximizing r can also be interpreted as maximizing the first derivative of the reliability of the predictions of the model [35].

4.3.3 Curiosity as changes in model predictions

The last way to model curiosity considers the idea developed in [31]. The idea is to bypass the predictions evaluator and express $r(t)$ as the difference between the model predictions changes directly. In other words, one defines $C(p_M(t), h(\leq\ t)) = \zeta(p_M(t), h(\leq\ t))$ and $g(a, b) = a - b$. In such a way, $r(t)$ becomes simply,

$$r(t) = \zeta(p_M(t-1), h(\leq t)) - \zeta(p_M(t), h(\leq t))\cdot \tag{4.11}$$

The curiosity expresses then directly the expected change in model predictions due to the learning algorithm. Assuming that running the learning algorithm leads necessarily to improvements of the model, $r(t)$ is expected to affect the agent in the same way as (4.10). Indeed, for parts of the environment that are already well modelled, the first and second term on the right side of (4.11) should remain approximately equal. Maximizing $r(t)$ leads to changes in predictions of the model before and after the learning algorithm did his job. Note however that this is synonym of improvements only under the assumption that the learning algorithm improves indeed the model.

4.4 Predicting the curiosity

The predictor p_R for the predictions of the curiosity cost has not been investigated yet. Formally, the future adaptive curiosity cost $r(t)$ can be computed as,

$$\sum_{\tau=t+1}^{\infty} r(t) = \sum_{\tau=t+1}^{\infty} \zeta(p_R, h(\tau)) \tag{4.12}$$

where ζ yields the predictions of p_R on the provided history h.

To obtain a good predictor p_R that can be used by the optimal controller, two problem arise. First, what tool or technique should be used such that good predictions can be achieved. Second, it may in practice be difficult to train such a predictor. Indeed, the dynamics of the car do not change over time since they depend solely on the physics of the environment. Which do not change over time. Whereas the curiosity cost does change over time because the model is adapting over time. Ideally, part of the environment that caught the attention of the controller beforehand will not require as many attention in the future. Because the model is improving. In other words, the interestingness of certain areas of the input-state space evolve over time. Which makes it challenging for the curiosity predictor to provide accurate results.

Similarly to the system identification problem, a linear regression or a neural network could be used to produce the predictions. While linear regression requires an explicit definitions of the features, neural networks have the advantage that they can learn in a much more general way. It is indeed difficult to carry out the same analysis about the feature dependencies on the intrinsic cost as it was done for the dynamics of the car.

Nevertheless, training the neural network for each time step again and again may be very expensive. This problem can in practice be solved by the fact the model is not likely to change much from time t to $t+1$ or $t+10$, if the learning-rate of the adaptive model learning algorithm is small enough. Consequently, from time t to $t+1$ one can train the neural network for a reasonable amount of gradient descent iterations by warm-starting it's parameters at time $t+1$ with the result of t using an appropriate optimization methods such as RMSprop for instance. In other words, the curiosity predictor is trained according to an incremental learning scheme.

Chapter 5

The controller

The present chapter explains how the controller of the race car is implemented. First, the formulation for optimal control problem associated with racing problem is described. The choice of the cost function is motivated together with the constraints on the inputs and outputs of the vehicle. Second, it is described how it is possible to take advantage of racing over a circuit to guarantee recursive feasibility of the problem and non-increasing performance for each completed lap. Third, it is presented how the curiosity term can be added to the cost function of the optimal control problem in order to obtain an active learning controller.

5.1 Problem formulation

Consider that the dynamics of the race car are given by,

$$z_{k+1} = f(z_k, u_k), \tag{5.1}$$

where $f : \mathbb{R}^{n_z} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_z}$, $z_k \in \mathbb{R}^{n_z}$ and $u_k \in \mathbb{R}^{n_u}$. Recall further that the states z and u of the vehicle are,

$$z = [x,\ y,\ \psi,\ v_x,\ v_y,\ \omega_z]^\top \quad \text{and} \quad u = [a_x,\ \delta]^\top. \tag{5.2}$$

In its simplest form, racing consists in travelling from the starting line to the finish line in a minimum amount of time. To achieve this goal, the driver tries to minimize the required time to reach the finish line. As the number of time steps to reach it is not known in advance, one way to solve the problem is to minimize the number of time steps required to complete the lap over an infinite horizon. Consider that the cost function associated to this problem is denoted by $l(z_k, u_k)$. To minimize the lap time, the cost function is expressed as,

$$l(z_k, u_k) = \|z_k - z_F\|^2, \tag{5.3}$$

where the final state of the race car on the finish line is denoted by z_F.

Consider further that the constraints on the states z are given by the set $\mathcal{Z}$,

$$\mathcal{Z} = \{z_k \in \mathbb{R}^{n_z} : z_L \leq z \leq z_U\}, \tag{5.4}$$

where for instance the distance of the vehicle y from the center-line of the road is subject to a constraint.

The sets $\mathcal{U}$ and $\Delta\mathcal{U}$ represent the constraints on the input and rate of change of the inputs,

$$\begin{aligned} \mathcal{U} &= \{u \in \mathbb{R}^{n_u} : u_L \leq u \leq u_U\}, & (5.5) \\ \Delta\mathcal{U} &= \{\Delta u \in \mathbb{R}^{n_u} : \Delta u_L \leq \Delta u \leq u_U\}\cdot & (5.6) \end{aligned}$$

The constraints on the inputs are namely the lower and upper bound on the steering δ and the acceleration a_x. The constraints on the rate of change of the inputs are necessary to prevent the controller from making the race car too aggressive.

In a general way, the racing car problem is expressed as,

$$\begin{aligned} J_{0\to\infty}(z(t)) = \quad & \min_u \sum_{k=0}^{\infty} l(z_k, u_k) \\ \text{subject to} \quad & \\ & z_{k+1} = f(z_k, u_k), \quad \forall\, k \geq 0 \\ & z_0 = z(t) \\ & z_k \in \mathcal{Z}, \quad \forall\, k \geq 0 \\ & u_k \in \mathcal{U}, \quad \Delta u_k \in \Delta\mathcal{U}, \quad \forall\, k \geq 0 \end{aligned} \quad . \tag{5.7}$$

Because the dynamics of the car a non-linear, the problem (5.7) is a non-linear optimal control problem. Therefore solving (5.7) over an infinite horizon is practically very difficult to achieve. The common way to address this problem is to solve the problem over a finite horizon. However, it is possible in the present case to solve the problem over the infinite horizon due to iterative nature of racing on a circuit.

5.2 Iterative learning control

Usually, racing from the start line to the finish line is completed in an iterative fashion where each iterate is referred to as a lap. As the race car completes several laps, the knowledge on the past visited states and inputs is collected and can be used by the controller for the next lap to increase the performances of the race car. This strategy of control is referred to as Iterative Learning Control (ILC). In order to solve (5.7) over an infinite horizon, it is possible to use the available knowledge of the completed laps to build the final safe set and a final cost [28].

5.2.1 Definitions and assumptions

Consider $z^j(t)$ and $u^j(t)$ as the states and inputs of the race car for lap j at time t. As the vehicle is completing lap j, the states and inputs are recorded and collected into $\mathbf{z}^j$ and $\mathbf{y}^j$ defined as,

$$\begin{aligned} \mathbf{z}^j &= [z^j(0), z^j(1), \ldots, z^j(t), \ldots] \\ \mathbf{u}^j &= [u^j(0), u^j(1), \ldots, u^j(t), \ldots] \end{aligned} \quad . \tag{5.8}$$

The first assumption from Rosolia et al. in [28] states that the closed loop trajectories of the race car should always start from the same initial state z_S for each lap j,

$$z^j(t=0) = z_S, \;\; \forall j \geq 0\cdot \tag{5.9}$$

The second assumptions regards the final state z_F of the closed-loop trajectories. Which should be the same for each lap j and who should be an equilibrium point of the system dynamics. In other words, the following must be satisfied,

$$z^j(t \to \infty) = z_F, \;\; \forall j \geq 0 \quad \text{and} \quad f(z_F, 0) = 0\cdot \tag{5.10}$$

The third assumption concerns the initial set of collected states and input of the system. Indeed, the build a safe final set and a final cost, a first feasible trajectory must be provided to the controller such that,

$$\mathbf{z}^0 \notin \emptyset, \quad \text{and} \quad \mathbf{u}^0 \notin \emptyset, \tag{5.11}$$

where $\mathbf{z}^0$ fulfils the first two assumptions, i.e. $z^0(t=0) = z_S$ and $z^0(t \to \infty) = z_F$.

To fulfil the three aforementioned assumptions in practice, note that the initial state z_S of the vehicle can be defined as the state of a stationary vehicle on the start line. Regarding the final state z_F, it can be arbitrary defined on or behind the finish line. Finally, the initial feasible trajectory can in practice be provided by another already reliable controller or a human driver for instance.

5.2.2 Final safe set

When the three assumptions discussed above are fulfilled, the final safe set can be computed from the samples of the previous feasible and successful trajectories. As it can practically be impossible to compute the final safe set for a non-linear problem [28], a sampled safe set is instead computed. The sampled safe set for lap j denoted by $\mathcal{SS}^j$ can be obtained according to,

$$\mathcal{SS}^j = \Big\{ \bigcup_{i \in \mathcal{M}^j} \bigcup_{k=0}^{\infty} z^j(t) \Big\}, \tag{5.12}$$

where $\mathcal{M}^j$ denotes the set of successful trajectory indexes before the j-th lap. In other words, the sample safe set is the collection of all states recorded for each time t of trajectory that reached the final state z_F. For the sake of simplicity, note that the terminology used in [28] is preserved.

From the definition of the second assumptions in (5.10), it follows that the set of successful trajectories M is simply defined as,

$$\mathcal{M}^j = \Big\{ l \in [0, j-1] : \lim_{t \to \infty} z^l(t) = z^F \Big\}\cdot \tag{5.13}$$

5.2.3 Final cost

The overall performance of the race car for each lap j can be computed from time $t = 0$ over the whole trajectory of visited states and inputs during the lap j as,

$$J^j_{0\to\infty} z^j(t=0) = \sum_{k=0}^{\infty} l(z^j_k, u^j_k)\cdot \tag{5.14}$$

The cost function of problem (5.7) express therefore the will to improve the performance of the vehicle from time t up to the finish line. Which can be expressed as,

$$J^j_{t\to\infty} z^j(t) = \sum_{k=t}^{\infty} l(z^j_k, u^j_k)\cdot \tag{5.15}$$

The sum over the infinite horizon can be split into two separate terms,

$$J^j_{t\to\infty} z^j(t) = \sum_{k=t}^{N-1} l(z^j_k, u^j_k) + \sum_{k=N}^{\infty} l(z^j_k, u^j_k) = J^j_{t\to N-1} z^j(t) + J^j_{N\to\infty} z^j(t), \tag{5.16}$$

where N denotes the length of the finite horizon.

As the first term can be computed the second cannot and must be approximated. The idea from [28] is to use the recorded samples from previous successful trajectories to compute the final cost $J^j_{N\to\infty}$. One defines the final cost p^j as,

$$p^j(z(t)) = \begin{cases} \min\limits_{(i,t)\in F^j(z(t))} & J^i_{t\to\infty}(z(t)), \quad \text{if} \quad z(t) \in \mathcal{SS}^j \\ & +\infty, \quad \text{if} \quad z(t) \notin \mathcal{SS}^j \end{cases} \tag{5.17}$$

where F is defined as,

$$F^j(z(t)) = \left\{(i,k) : i \in [0,j]\,, k \geq 0 \quad \text{and } \exists\, z^i_k \in \mathcal{SS}^j \quad \text{such that} \quad z^i_k = z(t)\right\}\cdot \tag{5.18}$$

The final cost $p^j(z(t))$ is computed from all the samples contained in $\mathcal{SS}^j$ and associates to $z(t)$ the feasible trajectory that represents the minimum cost to reach the final line. In other words, if the controller can reach a state $z(t)$ such that the associated final cost is not infinite, then there exist a feasible optimal trajectory to reach the final line.

5.2.4 Learning MPC problem

Using the knowledge of the sampled safe set $\mathcal{SS}^j$ and the final cost p^j, the problem (5.7) can be reformulated as,

$$
\begin{aligned}
J_{0\to\infty}(z(t)) = \quad & \min_{u} \; p^{j-1}(z_N) + \sum_{k=0}^{N-1} l(z_k, u_k) \\
\text{subject to} \quad & \\
& z_{k+1} = f(z_k, u_k), \quad \forall\, 0 \leq k \leq N \\
& z_0 = z(t) \\
& z_k \in \mathcal{Z}, \quad \forall\, 0 \leq k \leq N \\
& u_k \in \mathcal{U}, \;\; \Delta u_k \in \Delta\mathcal{U}, \quad \forall\, 0 \leq k \leq N \\
& z_N \in \mathcal{SS}^{j-1}
\end{aligned}
\quad . \tag{5.19}
$$

When the three aforementioned assumptions are fulfilled and under the further assumptions that the model of the race dynamics f is not subject to model mismatch, it can be shown [28] that for each lap j that the cost is non increasing and that the problem is recursively feasible. An non increasing cost means that the performance of the race car will improve for each new lap j or remain the same than the previous lap. Recursive feasibility means that that at any time and for each lap the constraints on the input and states are fulfilled and that the controller drives the car to the asymptotically stable equilibrium point z_F.

5.3 Incremental curiosity-driven learning model predictive control

In practice, there is always a model mismatch. In the previous chapters, it was argued that a better model can potentially lead the race car to better performances because it has a better knowledge of its environment. From the technical racing point of view, a better model may enable the controller to reach the boundaries of the "g-g" diagram. Furthermore, a better model brings us closer to the aforementioned assumption stating that problem (5.19) can be solve in a recursively feasible and non-increasing cost fashion.

One reformulates now (5.19) to provide the controller with the knowledge of the adaptive model F and the curiosity or intrinsic cost r,

$$
\begin{aligned}
J_{0\to\infty}(z(t)) = \quad & \min_{u} \; p^{j-1}(z_N) + \sum_{k=0}^{N-1} l(z_k, u_k) + \rho \sum_{k=0}^{N-1} r(z_k, u_k) \\
\text{subject to} \quad & \\
& z_{k+1} = F(z_k, u_k), \quad \forall\, 0 \leq k \leq N \\
& z_0 = z(t) \\
& z_k \in \mathcal{Z}, \quad \forall\, 0 \leq k \leq N \\
& u_k \in \mathcal{U}, \;\; \Delta u_k \in \Delta\mathcal{U}, \quad \forall\, 0 \leq k \leq N \\
& z_N \in \mathcal{SS}^{j-1}
\end{aligned}
\quad , \tag{5.20}
$$

where r is computed for the same horizon length N and ρ accounts for the exploration trade-off. It is indeed difficult to extend the theory of curiosity to the infinite horizon formulation. Ideally, one would expect r to decrease towards zero for an infinite horizon.

The race car may benefit of the curiosity cost in two distinct ways. First, as discussed in the third chapter, the curiosity cost drives ideally the race car towards more exciting states. Which improves the performances of the model. As the model mismatch is decreasing, the external cost of minimizing the lap time should therefore be improving. Eventually, the

curiosity cost tends to zero as the race car completes a high number of laps. Leading the race to a locally optimal trajectory.

Second, the curiosity term should steer the race car to previously unknown but more interesting states. According to the final sampled safe set, the controller allows these states to be visited only if a feasible and optimal trajectory can be established up to the finish line from all the previously recorded states. Therefore, the curiosity term may excite and extend the sampled safe set in such a way that the final cost can come up with a better trajectory, and therefore a better local optimum.

Finally, consider figure 5.1 which illustrates how the Active Learning Model Predictive Controller works. The input u_{k+1} and states z_{k+1} are recorded and given two the "History" module. Which manages the training batch for the adaptive model and the curiosity predictor. For the simplicity of the scheme, it is assume that the same module manages the necessary record samples to obtain the sampled final sampled safe set $\mathcal{SS}^{j-1}$ and the final cost p^{j-1}. Then, the system identification module computes is new best model $p_M(K+1)$. It communicates also with the curiosity model to produce the new curiosity predictor $p_R(k+1)$. This is done according to the scheme given in the previous chapter. Finally, these predictors and the sampled safe set and final cost are provided to the Learning Model Predictive Controller. Which solves problem (5.20) and applies the next optimal set of input to the race car.

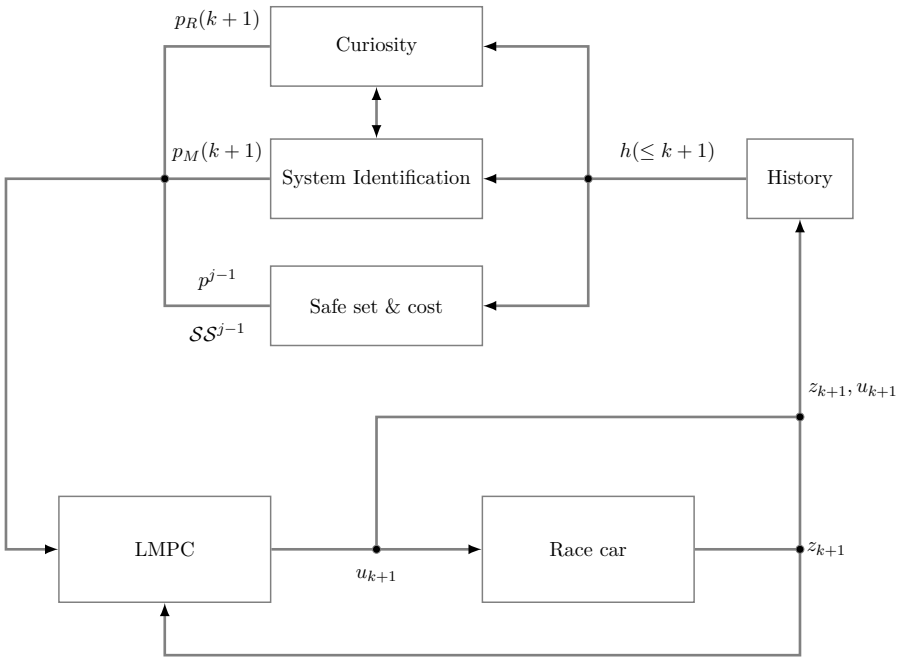


Figure 5.1: Illustration of the MPC controller

Chapter 6

Open-loop simulations

The first goal of this chapter is to find out which technique, linear regression of neural network, suits the best to identify the dynamics of the race car. For this purpose, two different models are built and cross-validated. The second goal is to determine the ability of the learning strategies to train the model for an open-loop simulation.

6.1 Cross-validation

The available data samples are provided from a simulation performed with the highly reliable industrial software `CarSim`. The vehicle is controlled with the LMPC controller described in the previous chapter, with the system identification proposed by Rosolia in [28]. The simulation was carried out on a portion of a the track where the car takes one 180 degrees corner. For each lap, the race car improves its performance. This is illustrated by the longitudinal velocity profile is illustrated in figure 6.1.

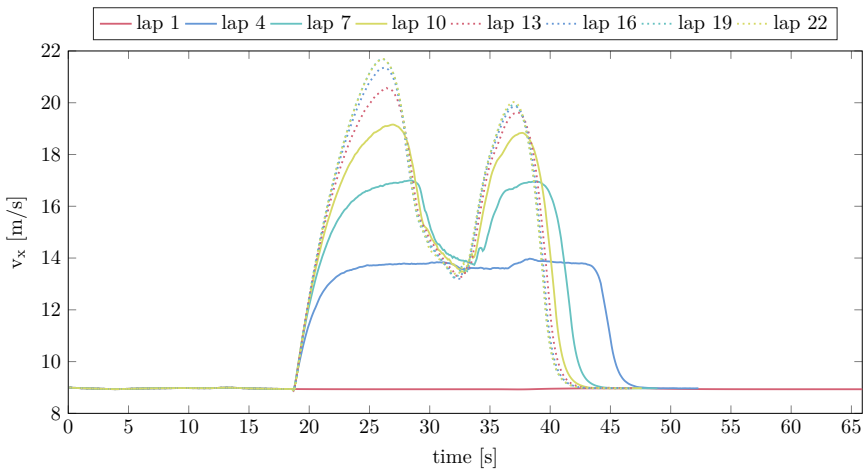


Figure 6.1: Illustration of the forward velocity v_x for several laps

For a given lap j, the car has the data from the previous completed laps $\leq j$ before starting the next lap $j+1$. For this reason, the goal of the system identification is to use the these recorded data and train a model F at predicting the dynamics of the race car for the next lap. As a reminder, the definition of F is given as,

$$F(X,\theta) = \begin{bmatrix} F_{v_x}(X,\theta_{v_x}) \\ F_{v_y}(X,\theta_{v_y}) \\ F_{\omega_z}(X,\theta_{\omega_y}) \end{bmatrix}, \tag{6.1}$$

and the goal of the system identification consists in choosing the best set of parameters θ_{v_x}, θ_{v_y} and θ_{ω_z}.

6.1.1 Cross-validation of the model

The cross-validation of the model is carried out for a specific lap j. This step consists in evaluating whether the features of the model have been wisely chosen or not. For the linear regression model, the choice for the features have been discussed in the second chapter. For the neural network model, the features are in fact defined by the number of units and hidden layers. The specific structure for the neural network model is defined below.

The model is cross-validated on lap $j = 10$. For the given data, lap 10 corresponds to a good trade-off between fast and changing dynamics. Indeed, from lap 10 to 11, the dynamics are changing significantly. In comparison, from lap 1 to 2 or from lap 21 to 22, the change is least noticeable and therefore less challenging.

For each model, the training curve, the learning curves and the validations curves are evaluated. For instance, the learning curves are helpful to evaluate the quality of the obtained model and know weather the model is prone to over-fitting or under-fitting. The validation curve helps to chosen the right regularization parameter λ. For the neural network model, a the use of a specific activation function and solver is also investigated prior of drawing these curves.

6.1.2 Cross-validation of the learning strategy

When the features of the model are established, the next step consists in evaluating weather or not the model can be used for incremental or semi-incremental learning. However, the procedure to assess the quality of incremental learning in general is not well defined. Because the batches containing the training and cross-validation examples are changing continuously, it is difficult to draw the learning and validations curves.

A simple way to asses the quality of the learning strategy consist in reproducing the simulation conditions from the first to the last lap of the available data set. For every lap j the model is trained according to the incremental or semi-incremental learning strategy. Before starting lap $j+1$, the performance of the resulting model is first evaluated and the same lap and recorded. Then the learning procedure moves forward and starts training the model on lap $j+1$, exactly as if the race car would be evolving in real-time. At the end of the learning procedure, it is then possible to evaluated the performance of the model on each lap j. This is the approach followed in the present work to cross-validate the learning strategy. In other words, the training examples are taken from the data of lap j and the cross-validation examples from lap $j+1$. One operates sequentially from lap 1 to 15. Note, that the model can further be evaluated on a test set given by lap $j+2$ for instance.

6.2 Linear regression

In this work, the features of the linear regression are defined as,

$$X = \Big\{1,\ v_x,\ v_y,\ \omega_z,\ a_x,\ \delta_f,\ v_x\omega_z,\ v_y\omega_z,\ \text{atan}(\frac{v_y}{v_x}),\ \frac{a_x}{v_x}\Big\}. \tag{6.2}$$

Note that additional features can eventually be added to the model. For instance, one did try to add the square of the longitudinal velocity to take the drag effects into account. However, the obtained results were less accurate for an unknown reason. It was therefore discarded.

6.2.1 Learning curves

The learnings curves for each model F_{v_x}, F_{v_y} and $F_{\omega_{zs}}$ are obtained as follows. The training batch starts with examples from the first lap. A model is trained with these examples and its performance is evaluated on the cross-validation data-set, i.e. lap 11 in the present case. Progressively, new training examples from the most recent laps are added until the training batch is made of all the training examples from lap 1 to 10.

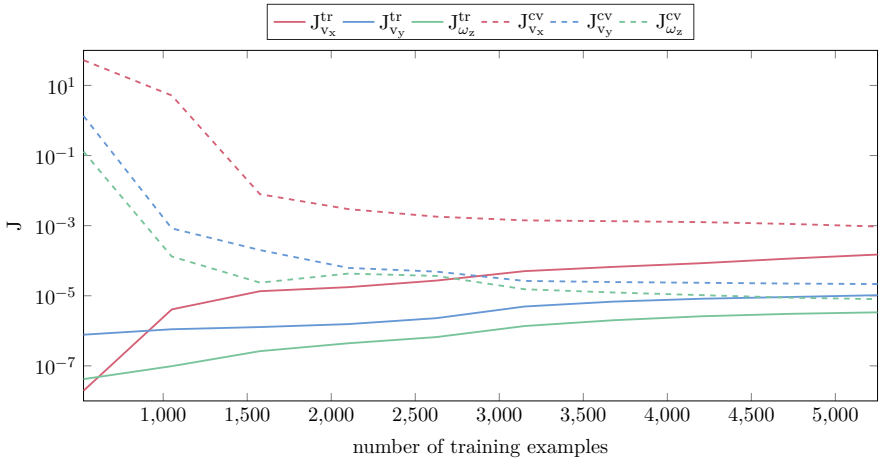


Figure 6.2: Learning curves for the linear regression model

The figure 6.2 show the results for the learning training curves of each model. The solid lines indicates the performance of the model on the training set and the dashed line the performances of the model on the cross-validation set. The figure shows in a first time that the features capture well the dynamics of the race car because the solid and dashed line seem the converge to the same point as the number of training examples increases. Which indicates that the model is not prone to under-fitting because the gap between each of them is relatively small. Second, the figure shows that adding more recent training examples results in better performances. As this result seems to be logical, it suggests implicitly that the old data can in fact be discarded from the training batch. For incremental learning application this is an important result. It suggests that the model would not require to be trained continuously on the old data samples because the most recent ones are the most determinant samples involved the accuracy of the model's predictions.

6.2.2 Validation curve

The validation curve is obtained by plotting the performance of the model evaluated on the training and cross-validation set for several values of the regularization parameter λ. The validation curve shows the influence of the regularization parameter on the over-fitting and under-fitting effects and helps therefore in choosing its right value for training the model.

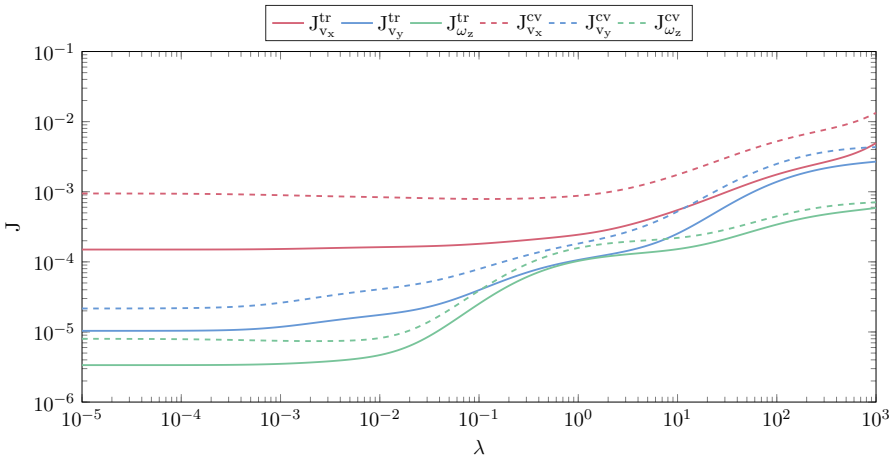
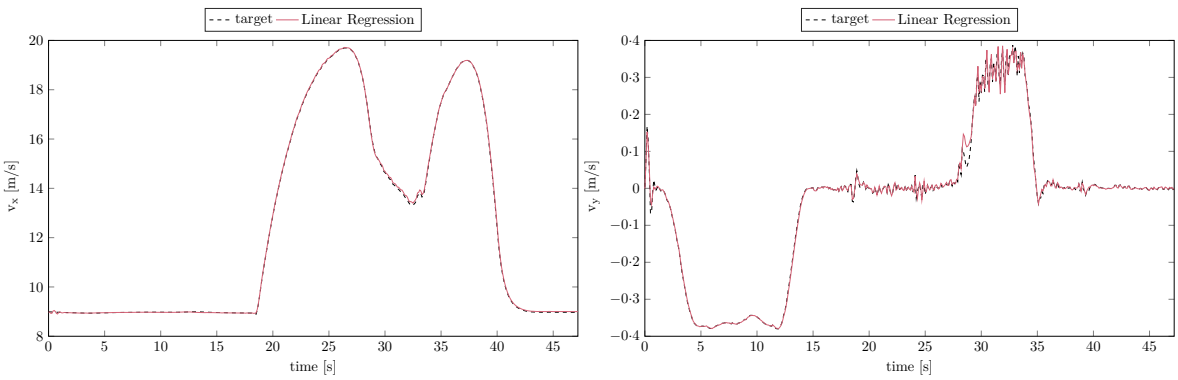


Figure 6.3: Validation curve for the linear regression model

Figure 6.3 shows that for any value of λ each of the three models are not prone to over-fitting. Whereas, for increasing values of λ, the model starts progressively to under-fit the training examples. For the three models, the regularization parameter can therefore be chosen to be arbitrarily small.

6.2.3 Simulation results

The figures 6.4a and 6.4b show the simulation results on the cross-validation set for the longitudinal and lateral velocity models. The dashed line shows the target and the red solid line indicates the prediction of the model. The results indicates that the dynamics of the race car can be well modelled using the linear regression technique. The results for the yaw rate model are not shown here, however the it performs similarly.



(a) Longitudinal velocity linear regression model F_{v_x}, open-loop simulation

(b) Lateral velocity linear regression model F_{v_y}, open-loop simulation

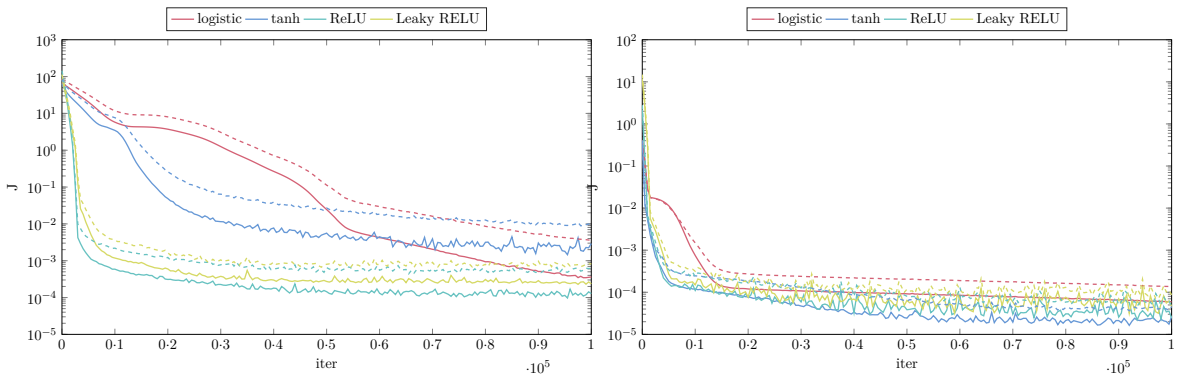
6.3 Neural network

The choice for the number of neurons and hidden layers for the neural network model is not a straightforward process and is often the result of a trial and error approach. It has been decided in the present work to choose a common network architecture for each of the three models. It is namely made of three hidden layers where each layer is composed of ten neurons. This choice is the result of a trade-off between accuracy and computational

efficiency. Due to time constraints, an architecture involving a high number of neurons may yield a model that is practically impossible to use for an optimal controller.

6.3.1 Choice of the sigmoid

The choice for the sigmoid function can be motivated through the visualization of the learning curves evaluated for each model and each sigmoid. The figures 6.5a and 6.5b show the learning curves obtained with various sigmoid functions for respectively the longitudinal and lateral velocity model. Note that the learning curves for the yaw rate behaves similarly to the results shown in 6.5b. The learning curves were obtained using the standard stochastic gradient descent method for fifty thousand iterations and a step size $\eta = 1 \cdot 10^{-4}$. Note that the dashed line on the figures show the training curve evaluated on the cross-validation set.



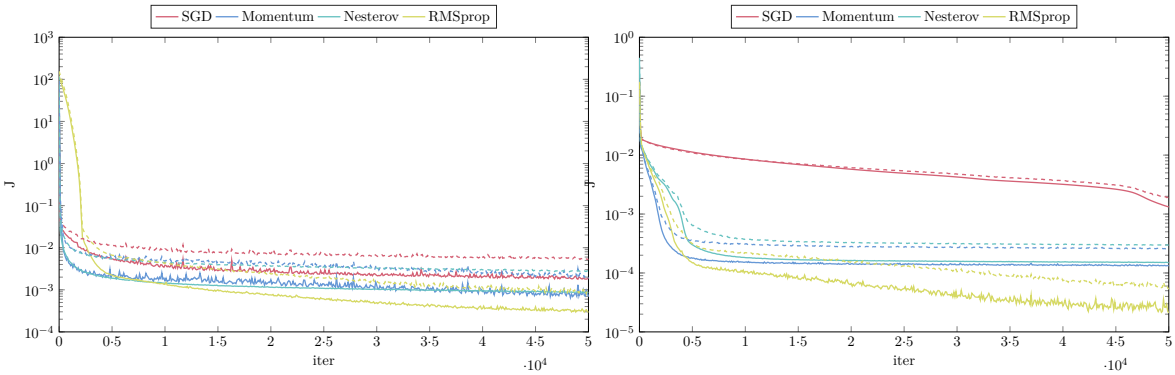
(a) Longitudinal velocity model, averaged training curve for different sigmoid functions

(b) Lateral velocity model, averaged training curve for different sigmoid functions

These results are interesting as they show that the accuracy of the model depends on the type of sigmoid function used. Figure 6.5a shows clearly that at neural network equipped with ReLU or leaky ReLU sigmoid performs better than using other types of sigmoid functions. Whereas 6.5b shows that the hyperbolic tangent sigmoid performs the best for the lateral velocity predictions. This may be explained by the fact this last is centred around the original and is bounded between -1 and $+1$. Whereas the ReLUs are not. The lateral velocity predictions, although not bounded, is usually centred around zero as well. Note that the yaw-rate predictions perform also better using the hyperbolic tangent sigmoid.

6.3.2 Choice of the solver

The choice of the solver is likely to play a important role in the accuracy of the model. In addition to the standard stochastic descent algorithm, three other solvers were described and discussed earlier in this work. The figures 6.6a and 6.6b show how the training curve behaves for the different solver. Note that these results were obtained starting systematically from the same initial point. The training curve is averaged over ten different learning curves. The parameters of the solver are defined by $\eta = 1 \cdot 10^{-4}$ and $\gamma = 0{\cdot}9$ for the all the solvers.



(a) Longitudinal velocity model, averaged training curve for different solvers

(b) Lateral velocity model, averaged training curve for different solvers

For both cases case the RMSprop method leads to a better local minimum. It is especially clear for the training of the lateral velocity model where RMSprop outperforms significantly the other methods. It is important to outline here that the obtained training curves for the yaw-rate model behaves similarly. The RMSprop method outperforms again the other methods. The reason why RMSprop yield significantly better results for the present application is obviously due to the fact that the learning rate is automatically tuned for each single parameter of the network whereas the other method keep it constant.

6.3.3 Learning curves

The learning curves for the neural network models are obtained in a similar way as the one obtained in the case of the linear regression. In view of the above obtained results, the model are trained using the RMSprop method. In addtion, F_{v_x} is made of ReLU sigmoid functions and both F_{v_y} and F_{ω_z} are made of hyperbolic tangent sigmoid functions. For each model, ten different learning curves are computed and the figure 6.7 show the averaged curves. Note that the number of iterations of the training algorithm is $50 \cdot 10^3$, $\eta = 1 \cdot 10^{-4}$ and $\gamma = 0{\cdot}9$.

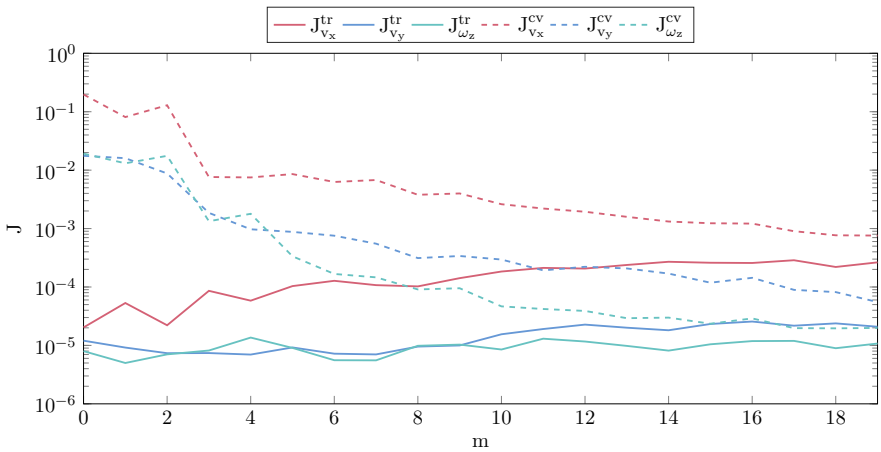


Figure 6.7: Learning curves for the neural network model

Interestingly, the learning curves for the neural network models shown on figure 6.7 are very similar to the curves obtained for the linear regression models. A similar analysis can therefore be carried out. It is yet important to highlight that the network architecture is appropriate in view of these results.

6.3.4 Validation Curve

The validation curves for the neural network models are obtained in a similar way as the one obtained in the case of the linear regression. The sigmoid functions, solver and solver parameters are the same as the one used to compute the learning curves.

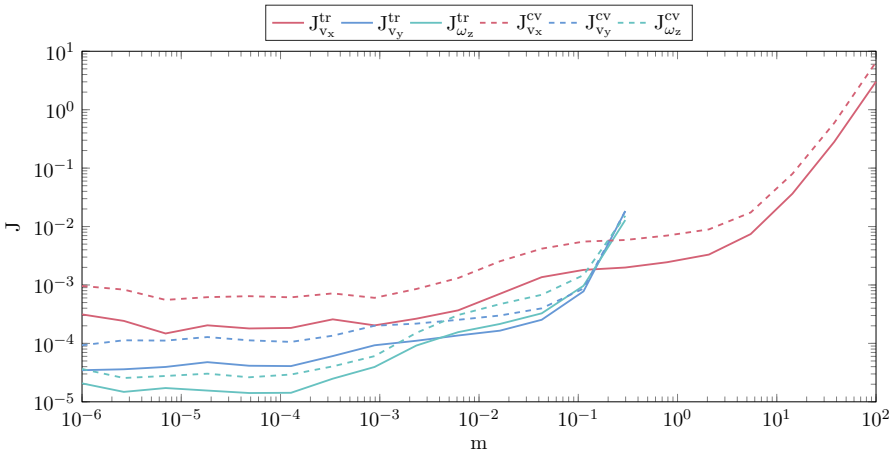


Figure 6.8: Validation curve for the neural network model

The figure 6.8 shows that the validation curves behaves also similarly to curves drawn in the case of the linear regression model. For increasing values of λ, the model is prone to under-fitting. Whereas small values of λ do not seem to cause over-fitting. In is interesting to notice that the model F_{v_x} seems to perform better on the cross-validation set for values of λ between 10^{-5} and 10^{-2}. For this reason, the regularization parameter for the longitudinal velocity prediction model is set to $\lambda 0{\cdot}001$ and λ is set arbitrarily small for the other models.

6.3.5 Simulation and comparison

The neural networks of each model are trained for $100 \cdot 10^3$ iterations using the RMSprop method, with a learning rate $\eta = 10^4$ and $\gamma = 0{\cdot}90$. The regularization parameters are set in view of the results from the validation curves analysis. For each model, the parameters of the neural networks are initialized with the appropriate initialization heuristic, which depends on the choice of the squashing function.

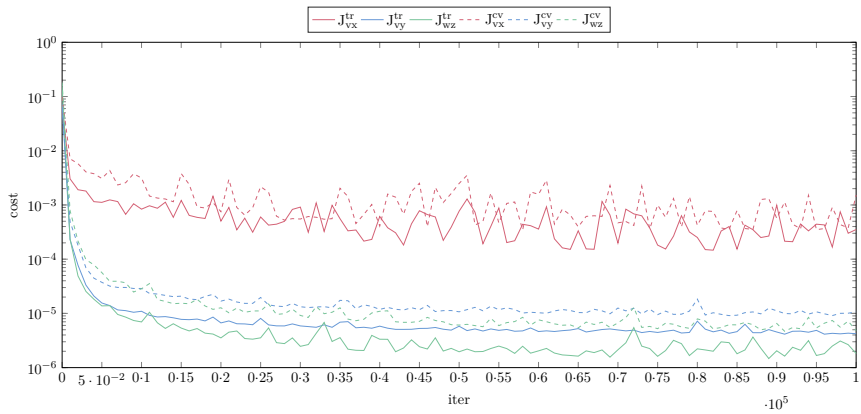
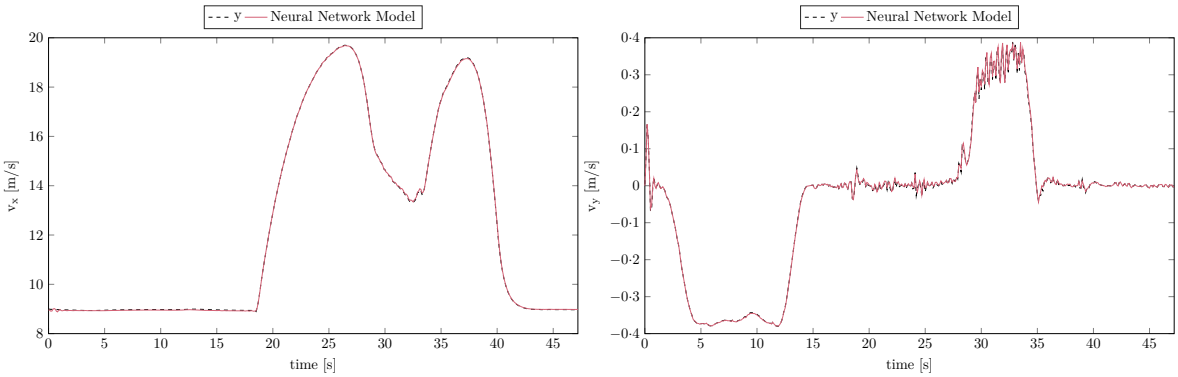


Figure 6.9: Training curve for the neural network models, open-loop simulation

	Linear regression	Neural Network
$\text{NRSME}_{\%,v_x}$	0.34	0.22
$\text{NRSME}_{\%,v_y}$	1.13	0.57
$\text{NRSME}_{\%,\omega_z}$	0.56	0.39

Table 6.1: Simulation performance measure

The figure 6.9 show the behaviour of the learning curve for each model during the training procedure. Notice first that F_{v_x} shows an overall higher residual cost than the models F_{v_y} and F_{ω_z}. This is simply due to the scale of the targets of the longitudinal velocity, around 15 m/s, which are much higher than the targets values for both the lateral velocity and yaw-rate, centred around zero. Second, the residuals are oscillating significantly, which suggests that the learning rate could eventually be decreased for this model. Then, while much progress in done for the first iterations, the residual do not decrease significantly over the last $50 \cdot 10^3$ iterations. Finally, it is important to highlight that the cost for the cross-validation set is generally non-increasing, which means that the models are not subject to over-fitting.



(a) Longitudinal velocity neural network model F_{v_x}, open-loop simulation

(b) Lateral velocity neural network model F_{v_y}, open-loop simulation

The figure 6.10a and 6.10b show respectively the results for the model F_{v_x} and F_{v_y}. These results show that neural networks can be used for the identification of the race car dynamics. In view of the training curves, the same results can be expected from the yaw-rate model, which are not shown here.

Finally, the results of the neural network and the linear regression model can be compared on basis of the Normalized Root-Mean Square Error (NRMSE). Tabular 6.1 compares the results obtained for both model. Although the performances of both models are relatively good, it shows namely that the neural network outperforms the linear regression model at predicting all the race car dynamics.

6.4 Learning strategy

The approach to cross-validate the learning strategy has been briefly introduced above. Nonetheless, further details about its implementation must be given. The incremental and semi-incremental strategies have in common that for each lap j, the performance of the model is evaluated on the cross-validation set, i.e lap $j + 1$. The performance can be evaluated by computing either the cost function on the data set or by evaluating the NRMSE. It has been decided to evaluate the NRMSE such that it can be compared to the results obtained below.

Starting from lap 1 to lap 15, a model is trained and its performance is evaluated for each lap. The result of this open-loop simulation indicates which learning strategy performs the best. For all the strategies, a linear regression and a neural network model are evaluated.

The training of the linear regression model for the semi-incremental strategy is easily implemented. Since the learning problem is convex, the result of training the model on lap j necessarily yields an optimal predictor of lap $j+1$. For the neural network model, the parameters of each model are once initialized according one of the heuristic rules for the first lap. Then, training the model on lap j is carried out by warm-starting the learning algorithm with the parameters of the model obtained for lap $j-1$. This makes it possible to train a neural network model with less than 10^4 iterations for each new lap instead of 10^5.

The advantage of the incremental learning strategy is that for each time step, an additional training sample is added to the training batch. Consequently, there is no need to wait for the next lap to incorporate that knowledge into the model. As new training examples are added to the training batch, the oldest are dropped such that the size of the training batch remains constant. For the present simulation it has been decided to keep the 500 most recent data samples in the training batch. The linear regression model is trained similarly to the semi-incremental strategy, except that a new model is computed for each new incoming data sample. Regarding the neural network model, the learning algorithm is warm-started for each time step with the parameter of the previous time step. In this way, the neural network model can be trained very efficiently with 20 iterations of gradient descent per time-step.

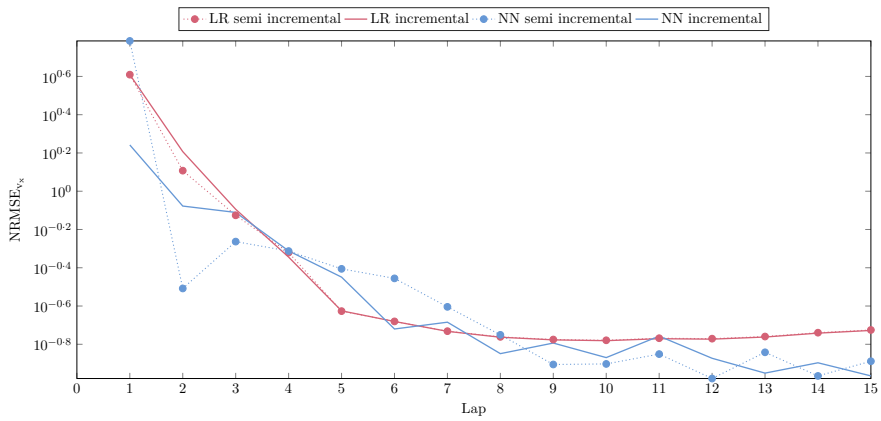
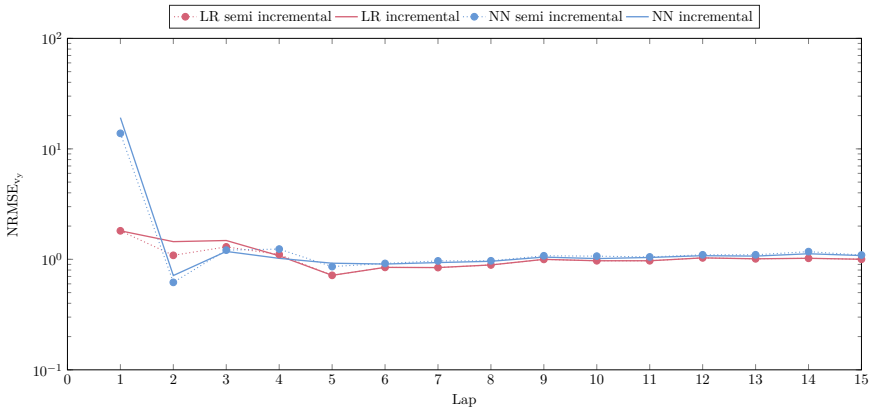
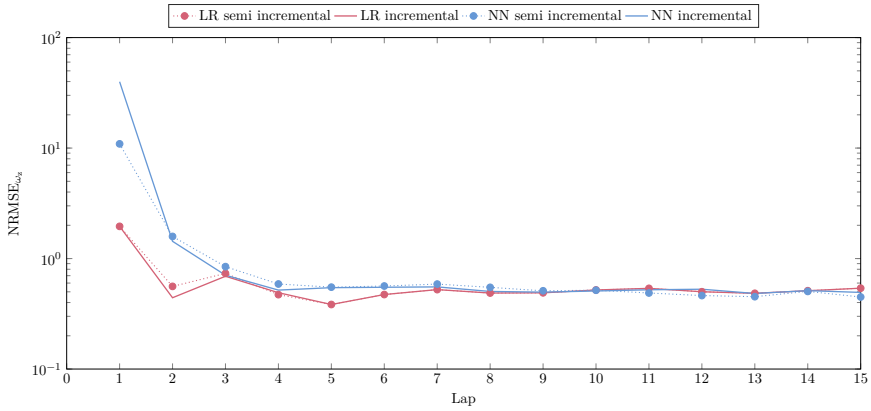


Figure 6.11: Learning strategy cross-validation for the longitudinal velocity model

The figure 6.11 shows the result of the simulation for the longitudinal velocity model. Notice first that the performance for the linear regression model yields almost the same results for the two training strategies. The neural network model trained with the semi-incremental learning strategy yields on average slightly better results than the linear regression model. When it is trained with the incremental learning strategy, surprisingly good results are obtained in the beginning while it needs more time from lap 4 to 8 to compete again with the other models.



(a) Learning strategy cross-validation for the lateral velocity model



(b) Learning strategy cross-validation for the yaw-rate model

Regarding the lateral velocity and yaw-rate model, the figures 6.12a and 6.12b show that the performance of the models is almost independent of the learning strategy. In fact, performing more training iterations on the neural network model is likely to yield better performances. However, for practical applications there is a trade-off between computational time efficiency and accuracy.

These open-loop simulation show that it is indeed possible to identify the dynamics of the vehicle in a semi-incremental and in an incremental fashion. It shows that training a neural network model can be carried out in real-time quite efficiently. For this reason and because the neural network model gives better predictions than the linear regression model, it was selected for the closed-loop simulations.

Chapter 7

Closed-loop simulations

This chapter presents the results obtained for the closed-loop simulations. Firstly, the simulation set-up and scenario and described. Second, the results for the two learning strategies are compared. Third, the curiosity cost is added to the cost function of the controller. The three different methods to generate the curiosity cost are tested, discussed and compared to the standard LMPC controller.

7.1 Simulation set-up

The simulations have been performed on a `Simulink` environment of `Matlab`. The high fidelity simulator `CarSim` is used as a plant model. Although many toolbox exist to train neural networks, the set-up is such that they are difficult to use and they cannot be used in the control design. For this reason, a neural network system based identification support has been implemented during the present project.

7.1.1 Description of the track

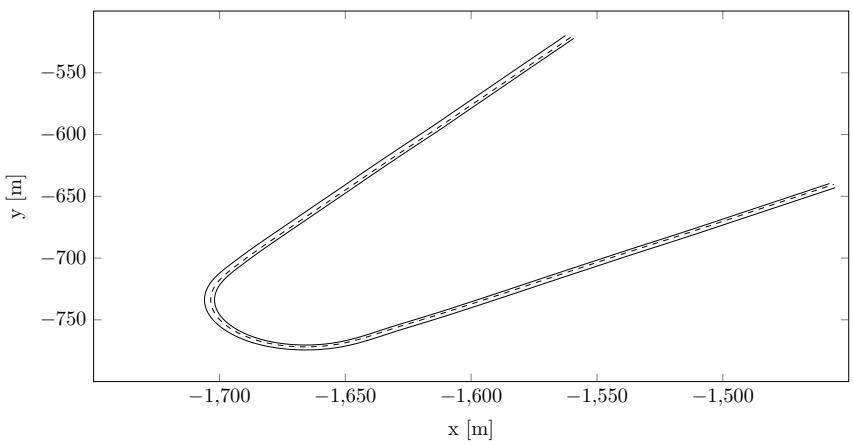


Figure 7.1: Illustration of the track used for the simulations

The race car simulation is performed on a small track illustrated on figure 7.1. The vehicle starts from the upper straight line, takes the corner and reaches the finish line through the lower straight line. Note that this corner, with a radius of approximately 20 m, is challenging

to take at high speed. This configuration is interesting because it is representative of the ability of the controller to manoeuvre the car in both straight-line and stiff corners.

A human race driver would first accelerate to gain speed on the first straight line. Before leaning into the corner, he would position the car towards the outside of the bend. In this way, he can reach the right apex at high speed. After the apex, he accelerates towards the center-line of the road to reach the finish line. In the straight lines, the longitudinal acceleration reaches its maximum, or minimum, in case of acceleration, or braking. At the apex, the lateral acceleration should be maximized such that the velocity can be kept high enough in the corner. Ideally, the autonomous race car should adopt the same behaviour.

7.1.2 Details for the controller

For each lap, the race car starts with the same initial state z_S and ends the lap with state z_F. They both fulfil the conditions imposed by the Learning Model Predictive Controller to guarantee recursive feasibility and a non-increasing cost. A path-following controller is used to steer the race car over the track such that a first feasible trajectory is provided to the controller.

The steering and acceleration are constrained as well as their rate of change. The value of the constraints on the acceleration is found to impact significantly the rate of change of the performance. Note that further simulations should be carried out for a longer horizon. The time step is equal to 0.12 s. For the each simulations, the length of the horizon is equal to 8. Note that further investigation will be carried out for a longer horizon.

7.1.3 Details for the predictors

The architecture for each neural network of the model remains the same as for the open-loop simulations, i.e 10 neurons for each of the 3 hidden layers. For the semi-incremental learning strategy, the model is re-trained after each lap is completed. The new model is then provided to the controller to complete the next lap. The incremental learning strategy uses every new incoming sample, store it in the training batch and re-train the model continuously. The training batch contains the 800 most recent samples and its size remains constant during the race. To train the models, both learning strategies use the the stochastic gradient descent algorithm with a mini-batch size of 50 samples. While the first strategy performs $50 \cdot 10^3$ iterations between each lap, the second strategy performs only 20 iterations per time step.

The curiosity predictor is made of an additional hidden layer of 10 neurons, i.e 4 in total. It has indeed been found that a deeper network performs better. And 4 hidden layers is a good trade-off between accuracy and computational efficiency. Since the models are always changing, the curiosity predictor is challenging to train. For this purpose, leaky RELUs have been chosen for the sigmoid functions. Although the learning algorithm is warm-started before each new training with the old parameters, it still requires more iterations than the race car model. For each time-step, the stochastic gradient descent method, RMSprop, performs 200 iterations with a mini-batch size of 20 samples.

7.2 Simulation for the learning strategy

To compare the two learning strategies, i.e incremental learning and semi-incremental learning, two different simulations of 20 laps each are carried out. The performances of the

controllers are first compared. Then, the evolution of the longitudinal velocity profile is evaluated.

7.2.1 Performance of the controller

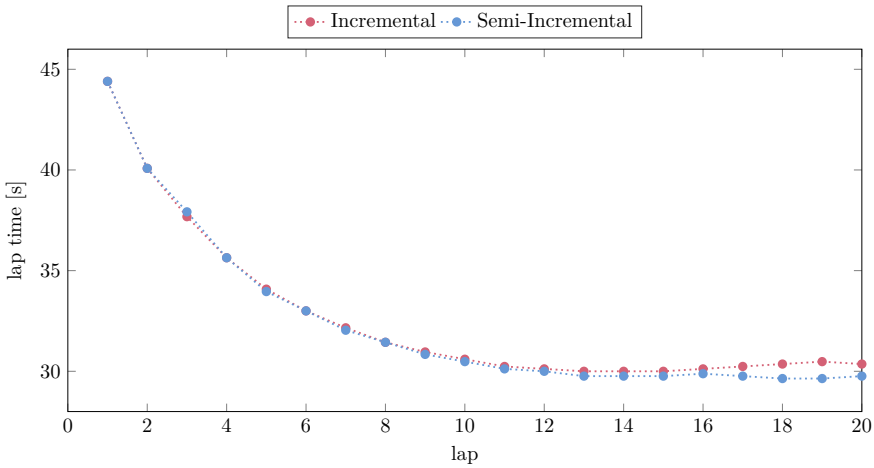
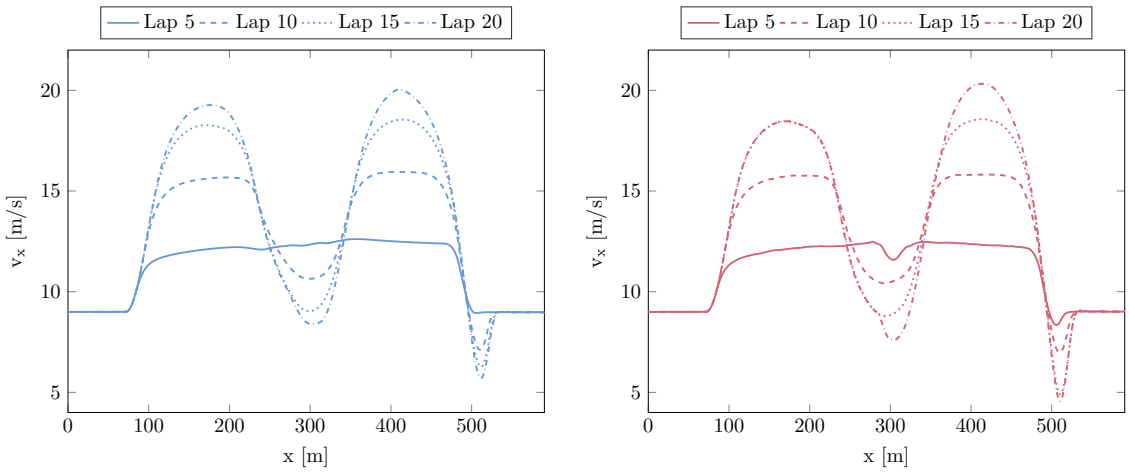


Figure 7.2: Performance of the controller for two learning strategies

Figure 7.2 shows the evolution of the lap time through the iterations. The performances of both controllers evolve in similar way during the first laps. Nevertheless, the semi-incremental strategy performs slightly better than the incremental strategy in the end of the simulation. It is important to notice that the cost is not non-decreasing. After 14 laps, the cost increases steadily for the second learning strategy. This is obviously due to model mismatch. Finally, the shape of the progress curves suggests that a local minimum is reached for the two different configurations. In the following we will see that the curiosity cost can overcome this problem.

7.2.2 Evolution of the velocity profile



(a) Evolution of the longitudinal velocity profile for the semi-incremental learning strategy

(b) Evolution of the longitudinal velocity profile for the incremental learning strategy

The figures 7.3b and 7.3a show the evolution of the speed profile of the race car for several laps. For both learning strategies, the speed increases normally in the straight lines and

decreases towards the apex of the corner. Nonetheless, the controller loses performances in the corner for each new lap. Whereas, it should always improve or at least reach some constant speed. In fact, it seems that the current configuration trades some speed increase in the straight-line against speed decrease in the corner. This problem is certainly due to model-mismatch.

7.3 Simulation for the curiosity cost

To evaluate the contribution of the curiosity cost on the controller, three simulations of 25 lap are performed. As a reminder, three different methods to generate the curiosity cost are proposed in the present work. The first method stimulates the controller to experience model mismatch. The second method encourages explicitly the controller to take trajectories that leads towards improvement of the model. The third one motivates the controller to improve its model implicitly. The objective here is to determine which method performs the best and compare it to the above obtained results.

7.3.1 Influence of active learning on the performance of the controller

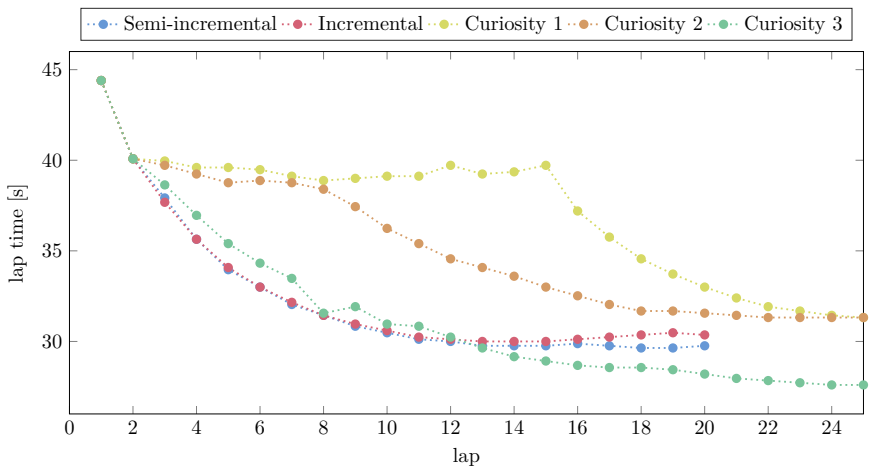


Figure 7.4: Comparison of the curiosity methods and learning methods, closed-loop simulation

The figure 7.4 shows clearly that the third method outperforms the two other methods. For the first one, the cost oscillates over the first laps and tends to decrease after. This suggests that the first method performs well with respect to its definition. Indeed, the controller appears to experience a high mismatch because of the oscillations. The second and third methods perform much better in the beginning. However, the improvement rate decreases at some point to reach, ultimately, similar performances as the first method.

The figure 7.4 shows also the obtained results without the curiosity cost. Using the third method to generate curiosity yields clearly better performances in the end. This result is very representative of one of active learning's main aspect. Indeed, during the first laps, the performance of the controller without curiosity is higher. After several laps, learning actively pays off. Which demonstrates that the controller is well able to balance the trade-off between exploration and exploitation. However, "*the curiosity may kill the cat*" [35]. In other words, active learning may yield worst results if this trade-off is not well balanced or, in the present case, if the curiosity predictions are poor.

7.3.2 Influence of active learning on the race-car behaviour

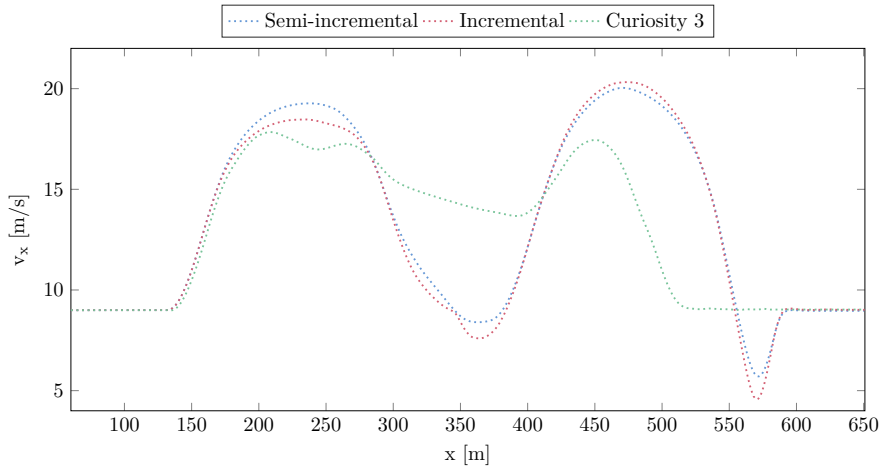


Figure 7.5: Longitudinal speed profile of the race car for the controller with and without active learning

The figure 7.5 compares the velocity profile of the third curiosity method to the two learning strategies without active learning. The speed profiles are drawn from the controller's best performances. Notice first that the race car with active learning is significantly slower on the straight-lines of the track. Whereas it is much faster in the corner. Which leads us to the two following statements. First, the active learning method would be truly able to decrease the model mismatch as the race knows it can lend into the bend a higher speed. Second, the active learning controller has still room for improvement in the straight-lines. Which leads us to the conclusion that the curiosity cost makes the controller able to escape from local minima.

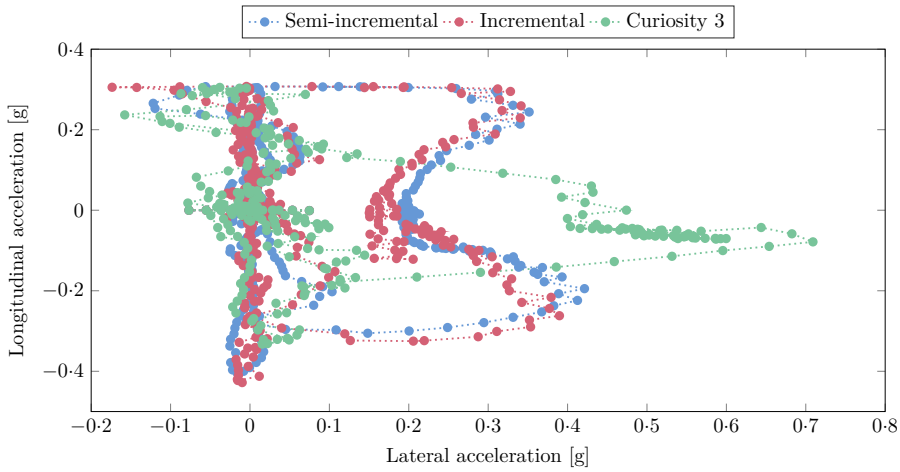


Figure 7.6: Illustration of the "g-g" diagram for the simulation with and without active learning

The figure 7.6 shows the "g-g" diagram for the three different trajectories. It shows namely that the controller with the curiosity exploits more efficiently the proficiency of the car at cornering. While the other simulations without curiosity shows that the car slows down in the curve. Which is clearly illustrated here. In other words, the controller is not aware of the car's ability to corner at a higher speed. Furthermore, the "g-g" diagram shows also that

the active learning controller can perform better in the straight-line. So there is still room for improvement.

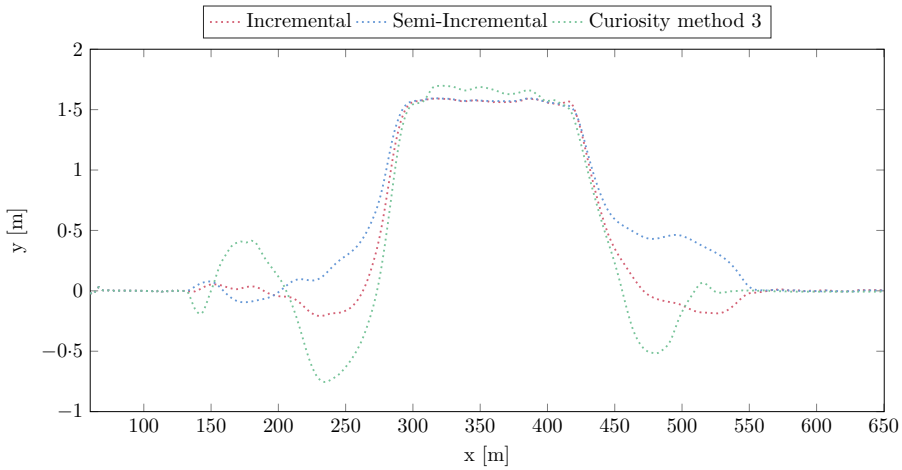


Figure 7.7: Lateral position of the race car on the track for the controller with and without active learning

Finally, the figure 7.7 shows the position of the vehicle from the center-line with respect to its position from the start-line. Positive values indicates that the race car is going towards the inner side of the bend. While the negative values indicate that is going towards the outer side of the corner. Interestingly, the active learning controller understands fundamental characteristics of racing that improve lap time. Whereas the controllers without curiosity do not seem that catch this subtlety. Indeed, figure 7.7 clearly illustrates the ability of the active learning controller to position the vehicle towards the outer part of the bend to reach the apex a higher speed. This is precisely the behaviour that a human race driver would adopt on this track.

Chapter 8

Conclusion

To conclude the present work, an Active Learning Model Predictive Controller has been proposed and successfully tested for the race car application. The ultimate goal of the work is to show how a model predictive controller can systematically improve the performance of the race car with respect to its past actions and through the acquisition of new knowledge on its environment. Reducing the model mismatch is critical to improve the performance of the race car and to ensure the controller recursive feasibility and non increasing cost. For this purpose, the present work has two main contributions. First, an efficient way to train a neural network model incrementally is proposed and successfully tested. Second, a curiosity cost is added to the optimal control objective function to perform an active system identification. In other words, the curiosity cost provides the controller a new skill that makes him able to determine the right trade-off between improving or exploiting the model. Which yields an Active Learning Model Predictive Controller.

For the purpose of system identification, two different methods were investigated. A linear regression and a neural network model have been successfully cross-validated for open loop simulations. While the first model has been easily implemented, the second one required more investigations. Especially regarding the choice of the optimization algorithm, the heuristic method for the parameter initialization and the sigmoid functions. It is found that both models are relatively insensitive to over-fitting. When compared, the neural network model outperforms the linear regression model.

Regarding the learning strategies, two different approaches are tested. The first identifies a new model after each lap. The second trains the model as soon as new data samples become available. While the linear regression model is computationally very cheap to train, this is not necessarily true for neural networks. Open loop simulations show that it is possible to train neural networks very efficiently for both learning strategies. There, the optimization algorithm is systematically warm-started with the parameters of the old model. Furthermore, it is found that the training can be carried out for each new data sample with only a few iterations of gradient descent. Which makes it an appealing learning strategy for real-time system identification. The closed loop simulations were carried out by combining a neural network model to the Learning Model Predictive Controller. Because the neural network performs better than the linear regression model and because an efficient learning strategy was found to train it.

During the closed-loop simulations, both learning strategies were tested. The results show that the race car learns to accelerate brake and corner. However, the controller was not pushing the car to the limits of the controllability. Better results are obtained when the curiosity cost is added to the objective function of the optimal controller. The three methods

for generating curiosity were tested and a deeper neural network is trained at forecasting future curiosity costs. For the third method, the performance of the race car increases significantly and outperforms clearly the other controllers. Furthermore, its behaviour on the track is much closer to that of a human driver on the same track.

Finally, the Active Learning Model Predictive Controller requires further improvements and investigations. But it is a first step towards another type of adaptive controller. This controller can eventually cover a very wide range of other problems. Problems where a reliable model of the system is required and where the acquisition of interesting data samples for the system identification is critical and difficult to achieve.

Reference

[1] P. Abbeel, V Ganapathi, and A. Y. Ng. Learning vehicular dynamics, with application to modelling helicopters. Technical report, Stanford University, 2005.

[2] A. Baranes and P. Y Oudeyer. Active learning of inverse models with intrinsically motivated goal exploration in robots. *Robotics and Autonomous Systems*, 2013.

[3] L. Bottou. Large-scale machine learning with stochastic gradient descent. In *Proc. of the International Conference on Computational Statistics*.

[4] S. Chen, S. A. Billings, and P. M. Grant. Non-linear system identification using neural networks. *International Journal of Control*, 1990.

[5] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control Signals and Systems*, 1989.

[6] A. Draeger, S. Engell, and H. Ranke. Model predictive control using neural networks. *IEEE Control Systems*, 1995.

[7] C. Giraud-Carrier. A note on the utility of incremental learning. *Artificial Intelligence Communications*, 2000.

[8] X. Glorot and Y. Bengio. Understanding the difficulty of training deep forward neural networks. In *Proc. of the 13th International conference on Artificial Intelligence and Statistics*.

[9] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. In *Proc. of the 14th International Conference on Artificial Intelligence and Statistics*.

[10] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *arXiv*, 2015.

[11] G. Hinton, N. Srivastava, and K. Swersky. Overview of mini-batch gradient descent. Course on Neural Networks for Machine Learning, lecture 6a.

[12] R. Houthooft, X. Chen, Y. Duan, J. Schulman, F. De Turck, and P. Abbeel. Curiosity-driven exploration in deep reinforcement learning via bayesian neural networks.

[13] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli. Kinematic and dynamic vehicle models for autonomous driving control design. Technical report, University of California at Berkeley and ETH Zürich, 2013.

[14] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural net- work. In *Proc. of the Neural Information Processing Systems Conference*.

[15] Y. LeCun, L. Bottou, G. B. Orr, and K. R. Müller. Efficient backprop. *Neural Networks: tricks of the trade*, 1998.

[16] Y. LeCun, I. Kanter, and S. A. Solla. Second-order properties of error surfaces: learning time and generalization. *Advances in Neural Information Processing Systems*, 1991.

[17] I. Lenz, R. Knepper, and A. Saxena. Deepmpc learning deep latent features for model predictive control. *Robotics Science and Systems*, 2015.

[18] M. Lopes, T. Lang, M. Toussaint, and P. Y. Oudeyer. Exploration in model-based reinforcement learning by empirically estimating learning progress. *Neural Information Processing Systems*, 2012.

[19] A. Maas, A. Y. Hannun, and A. Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. of the International Conference on Machine Learning*.

[20] J. Martens. Deep learning via hessian-free optimization. In *Proceding of the 27th International Conference on Machine Learning*.

[21] William F. Milliken and Milliken Douglas L. *Race car vehicle dynamics*. IMechE, 1995.

[22] Y. Nesterov. A method for unconstrained convex minimization problem with the rate of convergence 0(1/k2). *Doklady ANSSSR(translated as Soviet.Math.Docl)*, 1983.

[23] A. Y. Ng. Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proc. of the 21st International Conference on Machine Learning*.

[24] P. Y. Oudeyer, Kaplan F., and V. V. Hafner. Intrinsic motivation systems for autonomous mental development. *Transactions on evolutionary computation*, 2007.

[25] A. Punjani and P. Abbeel. Deep learning helicopter dynamics models. Technical report, University of California at Berkeley, 2013.

[26] N. Qian. On the momentum term in gradient descent learning algorithms. *The Official Journal of the International Neural Network Sociey*, 1999.

[27] R. Rajamani. *Vehicle Dynamics and Control*. Springer, 2011.

[28] Ugo Rosolia and F. Borrelli. Learning non-linear model predictive control for iterative tasks. Technical report, University of California at Berkeley, 2016.

[29] S. Ruder. An overview of gradient descent algorithms, 2015.

[30] H. Sakai. The dynamic properties of tires. *International Journal of Vehicle Design*, 1981.

[31] J. Schmidhuber. Curious model-building control systems. In *In Proc. International Joint Conference on Neural Networks*.

[32] J. Schmidhuber. A possibility for implementing curiosity and boredom in model-building neural controllers. In *Proc. of the International Conference on Simulation of Adaptive Behavior: From Animals to Animats*.

[33] J. Schmidhuber. Dynamische neuronale netze und das fundamentale raumzeitliche lernproblem. Technical report, Institut für Informatik, Technische Universität München, 1990.

[34] J. Schmidhuber. Developmental robotics, optimal artificial curiosity, creativity, music, and the fine arts. *Connection Science*, 2006.

[35] J. Schmidhuber. Formal theory of creativity, fun, and intrinsic motivation (1990-2010). *IEEE Transactions on Autonomous Mental Development*, 2010.

[36] J. Schmidhuber, J. Storck, and J. Hochreiter. Reinforcement driven information acquisition in non-deterministic environments. In *In Proc. ICANN'95*.

[37] B. Settles. Active learning literature survey. Technical report, University of Wisconsin-Madison, 2010.

[38] S. Spielberg, Z. C. Peng, and T. K. Shen. Control of dynamic systems using lstm supported neural network. 2016.

[39] Y. Sun, F. Gomez, and J. Schmidhuber. Planning to be surprised: Optimal bayesian exploration in dynamics environments. *Artificial General Intelligence*, 2011.

[40] R. S. Sutton. Two problems with backpropagation and other steepest-descent learning procedures for networks. In *Proc. 8th Annual Conf. Cognitive Science Society*.

[41] J. M. Twomey and A. E. Smith. Performance measures, consistency, and power for artificilial neural networks models. *Mathl. Comput. Modelling*, 1995.

[42] B. Xu, N. Wang, T. Chen, and M. Li. Empirical evaluation of rectified activations in convolution network. *arXiv preprint arXiv:1505.00853*, 2015.

[43] T. Zhang, G. Kahn, S. Levine, and P. Abbeel. Learning deep control policies for autonomous aerial vehicles with mpc-guided policy search. In *International Conference on Robotics and Automation (ICRA)*.